

## Web Technology

- Web technology is the use of various tools and techniques to create, design, develop, and maintain websites and web applications on the internet.
- Web technology consists of different components, such as web browsers, web servers, web protocols, web languages, web frameworks, web databases, and web APIs.
- Web browsers are software applications that allow users to access and view web pages on the internet. Examples of web browsers are Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- Web servers are software applications that host and deliver web pages and web resources to web browsers. Examples of web servers are Apache, Nginx, IIS, and Tomcat.
- Web protocols are rules and standards that define how web browsers and web servers communicate and exchange data over the internet. Examples of web protocols are HTTP, HTTPS, FTP, and SMTP.
- Web languages are programming languages that are used to create and manipulate web pages and web applications. Examples of web languages are HTML, CSS, JavaScript, PHP, and Python.
- Web frameworks are software libraries that provide common functionalities and features for web development. Examples of web frameworks are Bootstrap, jQuery, Angular, Laravel, and Django.
- Web databases are software applications that store and manage data for web applications. Examples of web databases are MySQL, MongoDB, PostgreSQL, and SQLite.
- Web APIs are interfaces that allow web applications to interact with other web services or applications. Examples of web APIs are RESTful API, SOAP API, GraphQL API, and WebSockets API.

### Unit 1 - Introduction to Web Technology

- Web technology is the use of various tools and protocols to create and deliver information and services over the internet or a network of computers.
- Web technology consists of two main components: the client and the server.
  - The client is the device or software that requests and displays web content, such as a web browser, a mobile app, or a smart TV.
  - The server is the device or software that responds to the client's requests and provides web content, such as a web server, a database, or an application server.
- Web technology enables various types of web applications, such as websites, web services, web portals, web games, and web-based software.
- Web technology is based on a set of standards and protocols that define how web content is formatted, transmitted, and processed.
  - Some of the common web standards and protocols are:
    - \* HTML (HyperText Markup Language): the language that de-



## Introduction to Web Technology

- Web technology is the development of the mechanism that allows two or more computer devices to communicate over a network.
- Web technology includes markup languages, programming interfaces, standards to define document identity and display.
- Web technology enables the creation and delivery of web pages, which are documents that contain text, data, pictures, animation, and video on the Internet.
- Web technology also facilitates the interaction between web servers and clients, which are programs that request and display web pages.
- Web technology is constantly evolving and improving, with new trends and features emerging every year.
- Web technology is important because it enables the exchange of information, services, and resources across the world, enhancing communication, collaboration, and innovation.

## Web Development Strategies

Web development strategies are the methods and techniques used in the planning, design, development, deployment and management of web applications and systems. The aim of web development strategies is to ensure that these applications and systems are aligned with the business goals of the organisation, and that they meet the needs of the users.

Some of the web development strategies are:

- **Define the purpose and objectives of the web application or system.** This involves identifying the target audience, the value proposition, the core features and functionalities, and the expected outcomes of the web application or system .
- **Choose the appropriate technologies and tools for the web development project.** This involves selecting the programming languages, frameworks, libraries, databases, hosting services, and other tools that are suitable for the web development project, based on the requirements, budget, timeline, and scalability of the web application or system .
- **Design the user interface and user experience of the web application or system.** This involves creating the layout, navigation, colour scheme, typography, graphics, and other elements that make up the visual and interactive aspects of the web application or system. The design should be user-friendly, responsive, accessible, and consistent across different devices and browsers .
- **Develop the web application or system using the chosen technologies and tools.** This involves writing the code, testing the functionality, debugging the errors, and integrating the different components of the web application or system. The development process should follow the best practices and standards of web development, such as code quality,

security, performance, and documentation .

- **Deploy and manage the web application or system.** This involves launching the web application or system to the public, monitoring its performance, updating its content and features, and maintaining its security and reliability. The deployment and management process should also include feedback collection, analytics, and evaluation of the web application or system’s effectiveness and user satisfaction .

These are some of the web development strategies that can help you create a successful web application or system. However, these strategies are not fixed or universal, and they may vary depending on the specific needs and goals of each web development project. Therefore, it is important to conduct a thorough research and analysis of the web development project before applying any of these strategies.

## History of Web

- The World Wide Web (WWW) is a system of interconnected documents that can be accessed through the Internet using a web browser.
- The Web was invented in 1989 by Tim Berners-Lee, a British scientist working at CERN, a European research organization based in Geneva, Switzerland .
- Berners-Lee proposed a “universal linked information system” that used several concepts and technologies, such as hypertext, Uniform Resource Identifier (URI), HyperText Transfer Protocol (HTTP), and HyperText Markup Language (HTML).
- By the end of 1990, the first web page was served on the open internet, and in 1991, people outside of CERN were invited to join this new web community.
- The Web grew rapidly in the 1990s, with the emergence of popular web browsers, such as Mosaic, Netscape, and Internet Explorer, and web services, such as Yahoo, Amazon, and eBay.
- The Web also enabled new forms of communication, collaboration, and creativity, such as email, social media, wikis, blogs, podcasts, and web applications.
- The Web has evolved over time, with the development of new standards, protocols, and technologies, such as Cascading Style Sheets (CSS), JavaScript, Extensible Markup Language (XML), Web Services, Web 2.0, Semantic Web, and Web 3.0.
- The Web has become a global public resource that is used by billions of people for various purposes, such as education, entertainment, commerce, and social good.
- The Web also faces many challenges, such as security, privacy, accessibility, digital divide, misinformation, and regulation.

## History of Internet

- The internet is a system of interconnected computer networks that allows communication and data exchange across the world.
- The origins of the internet can be traced back to the 1950s, when the United States government funded research projects to develop technologies for military and scientific purposes during the Cold War.
- One of the key projects was the Advanced Research Projects Agency Network (ARPANET), which was launched in 1969 and connected four universities in the US. ARPANET was the first network to use packet switching, a method of transmitting data in small units called packets.
- Another important project was the International Network Working Group (INWG), which was formed in 1972 and proposed the idea of a global network of networks based on open standards and protocols. The INWG developed the Transmission Control Protocol (TCP) and the Internet Protocol (IP), which are the core protocols of the internet today.
- The term “internet” was first used in 1974 by Bob Kahn and Vint Cerf, who are considered the fathers of the internet. They published a paper describing the design and implementation of TCP/IP, which enabled different networks to communicate with each other.
- The internet gradually expanded and evolved over the decades, with the introduction of new technologies and applications, such as email, domain name system (DNS), hypertext, World Wide Web (WWW), browsers, search engines, social media, e-commerce, and cloud computing.
- The internet has become a global phenomenon that has transformed various aspects of human society, such as communication, education, entertainment, business, politics, and culture. It has also raised new challenges and issues, such as cybersecurity, privacy, digital divide, and online misinformation.

## Protocols Governing Web

- Protocols are a set of rules and standards that enable communication between different devices and applications on a network.
- The web is a collection of interconnected documents and resources that are accessed through the internet using web browsers and web servers.
- The web relies on various protocols to function properly and efficiently. Some of the most common protocols governing the web are:
  - **TCP/IP (Transmission Control Protocol/Internet Protocol):** This is the fundamental protocol suite that provides the basic functionality of the internet. It consists of two main protocols: TCP and IP. TCP is responsible for ensuring reliable and ordered delivery of data packets between hosts, while IP is responsible for routing and addressing the packets across the network. TCP/IP also supports other protocols such as UDP, ICMP, ARP, etc.

- **HTTP (HyperText Transfer Protocol):** This is the protocol that defines how web browsers and web servers communicate and exchange information. HTTP uses a request-response model, where a client (browser) sends a request for a resource (such as a web page, an image, a video, etc.) to a server, and the server responds with the requested resource or an error message. HTTP also supports methods such as GET, POST, PUT, DELETE, etc. to perform different actions on the resources. HTTP is based on TCP/IP and uses port 80 by default.
- **DNS (Domain Name System):** This is the protocol that translates human-readable domain names (such as `www.google.com`) into numerical IP addresses (such as `172.217.14.206`) that are used by TCP/IP to locate and connect to the hosts. DNS uses a hierarchical and distributed database of name servers that store and resolve the domain name queries. DNS also supports other types of records such as MX, CNAME, NS, etc. to provide additional information about the domains. DNS is based on UDP/IP and uses port 53 by default.
- **FTP (File Transfer Protocol):** This is the protocol that enables the transfer of files between hosts on a network. FTP uses a client-server model, where a client (FTP client) initiates a connection to a server (FTP server) and requests to upload or download files from the server. FTP also supports commands such as LIST, RETR, STOR, DELE, etc. to manipulate the files and directories on the server. FTP is based on TCP/IP and uses port 21 by default.

## Writing Web Projects

Writing web projects is a skill that involves planning, designing, developing, testing, and deploying websites or web applications. Web projects can range from simple static pages to complex dynamic applications that interact with databases, APIs, and other services. Web projects can also be used for various purposes, such as personal portfolios, blogs, e-commerce, social media, education, entertainment, etc.

Some of the steps involved in writing web projects are:

- **Researching the requirements:** This involves understanding the goals, audience, scope, and features of the web project. It also involves gathering information from the client, stakeholders, users, competitors, and existing solutions. Researching the requirements helps to define the problem, the solution, and the value proposition of the web project.
- **Identifying the project phases and activities:** This involves breaking down the web project into manageable and measurable tasks, such as design, development, testing, deployment, and maintenance. It also involves estimating the time, cost, and resources needed for each task, and setting

milestones and deadlines. Identifying the project phases and activities helps to organize the work, track the progress, and manage the risks of the web project.

- **Choosing the technologies and tools:** This involves selecting the appropriate languages, frameworks, libraries, databases, APIs, and other tools that will be used to create the web project. It also involves setting up the development environment, such as the code editor, the version control system, the testing tools, the hosting service, etc. Choosing the technologies and tools helps to ensure the functionality, performance, security, and compatibility of the web project.
- **Creating the design and the content:** This involves creating the visual and textual elements of the web project, such as the layout, the color scheme, the typography, the images, the icons, the logo, the navigation, the forms, the buttons, the headings, the paragraphs, etc. It also involves applying the principles of user interface design, user experience design, and web accessibility. Creating the design and the content helps to communicate the message, the brand, and the value of the web project.
- **Writing the code and the documentation:** This involves writing the HTML, CSS, and JavaScript code that will define the structure, style, and behavior of the web project. It also involves writing the backend code that will handle the data, the logic, and the communication of the web project. Additionally, it involves writing the documentation that will explain the purpose, the features, the installation, the usage, and the contribution of the web project. Writing the code and the documentation helps to implement the solution, the functionality, and the quality of the web project.
- **Testing and debugging:** This involves checking the web project for errors, bugs, and issues that may affect its functionality, performance, security, or usability. It also involves fixing the errors, bugs, and issues that are found, and verifying that the web project meets the requirements and the expectations. Testing and debugging helps to ensure the reliability, the efficiency, and the satisfaction of the web project.
- **Deploying and maintaining:** This involves uploading the web project to a server, a hosting service, or a platform that will make it accessible to the public. It also involves updating, monitoring, and improving the web project as needed, based on the feedback, the analytics, and the changes in the environment. Deploying and maintaining helps to deliver, promote, and sustain the web project.

Writing web projects is a challenging but rewarding process that requires creativity, problem-solving, and technical skills. By following these steps, you can create web projects that are useful, attractive, and successful.

## Connecting to Internet

- The Internet is a global network of computers and devices that can communicate and share information with each other.
- To connect to the Internet, you need an Internet service provider (ISP) that provides you with access to the network, and a device that can send and receive data over the network.
- There are different types of Internet connections, such as Wi-Fi, Ethernet, broadband, satellite, and dial-up. Each type has its own advantages and disadvantages, such as speed, reliability, cost, and availability.
- To connect to the Internet using Wi-Fi, you need a wireless router that broadcasts a Wi-Fi signal, and a device that has a Wi-Fi adapter that can detect and join the signal. You may also need a password to access the Wi-Fi network, depending on the security settings of the router.
- To connect to the Internet using Ethernet, you need an Ethernet cable that connects your device to a modem or a router that is connected to the Internet. Ethernet cables provide a faster and more stable connection than Wi-Fi, but they limit the mobility of your device.
- To connect to the Internet using broadband or satellite, you need a modem that connects to a cable or satellite dish that receives the Internet signal from your ISP. Broadband and satellite connections offer high-speed Internet access, but they may not be available in some areas or may be affected by weather conditions.
- To connect to the Internet using dial-up, you need a modem that connects to a phone line that dials a number to access the Internet. Dial-up connections are slow and unreliable, but they are cheap and widely available.
- The steps to connect to the Internet may vary depending on your device, operating system, and type of connection. Here are some general steps for some common devices and connections:
  - Windows 10 Wi-Fi: Select the Network icon on the taskbar, choose the Wi-Fi network you want, then select Connect. Type the network password, and then select Next. Choose Yes or No, depending on the type of network you're connecting to and if you want your PC to be discoverable by other PCs and devices on the network.
  - Windows 10 Ethernet: Plug one end of the Ethernet cable into your device and the other end into the modem or router. Wait for the Network icon on the taskbar to show a connected status.
  - Android Wi-Fi: Go to Settings > Network & internet > Internet to turn on Wi-Fi. Make sure that the Wi-Fi connection for your device is on. Tap the name of the Wi-Fi network you want to join. If prompted, enter the password and tap Connect.



- iOS Wi-Fi: Go to Settings > Wi-Fi to turn on Wi-Fi. Tap the name of the Wi-Fi network you want to join. If prompted, enter the password and tap Join.
- Mac Wi-Fi: Click the Wi-Fi icon in the menu bar, then choose a network. If prompted, enter the password and click Join.
- Mac Ethernet: Plug one end of the Ethernet cable into your device and the other end into the modem or router. Wait for the Wi-Fi icon in the menu bar to change to an Ethernet icon.

### **Introduction to Internet services**

- Internet services are the applications or functions that allow us to access, exchange, or obtain information over the internet, which is a global system of interconnected computer networks.
- Internet services can be classified into four main categories: communication services, information retrieval services, web services, and video conferencing services.
- Communication services enable us to communicate with individuals or groups using various methods, such as email, instant messaging, chat, voice over IP, and social media.
- Information retrieval services allow us to search, access, and download data or files from the internet, such as text, graphics, sound, and software. Some examples of information retrieval services are search engines, online databases, and file transfer protocol.
- Web services are software applications that run on the internet and provide functionality or data to other applications or users. Web services use standard protocols, such as HTTP and XML, to communicate and exchange data. Some examples of web services are online shopping, online banking, and online gaming.
- Video conferencing services enable us to have real-time audio and video communication with one or more people over the internet. Video conferencing services use specialized software and hardware to capture, transmit, and display video and audio signals. Some examples of video conferencing services are Skype, Zoom, and Google Meet.

### **Introduction to Internet Tools**

- Internet tools are programs, applications, or technologies that can be accessed via an internet connection and enhance the communication, collaboration, or learning of internet users .
- Some examples of internet tools are:
  - Web browsers: software applications that allow users to access and view web pages on the internet, such as Google Chrome, Mozilla Firefox, or Microsoft Edge.
  - Search engines: websites or applications that allow users to find information on the internet by entering keywords or queries, such as

- Google, Bing, or Yahoo.
- Email: a system that allows users to send and receive electronic messages over the internet, such as Gmail, Outlook, or Yahoo Mail.
- Social media: websites or applications that allow users to create and share content, such as text, images, videos, or audio, and interact with other users, such as Facebook, Twitter, or Instagram.
- Online learning: websites or applications that allow users to access educational content, such as courses, lectures, quizzes, or assignments, and learn at their own pace, such as Coursera, Khan Academy, or Udemy.
- Cloud computing: a service that allows users to store, access, and process data and applications on remote servers over the internet, rather than on their own devices, such as Google Drive, Dropbox, or Microsoft Azure.
- E-commerce: a system that allows users to buy and sell goods and services over the internet, such as Amazon, eBay, or Shopify.
- Online banking: a service that allows users to manage their financial transactions and accounts over the internet, such as PayPal, Venmo, or Zelle.
- Streaming: a technology that allows users to watch or listen to media content over the internet, such as Netflix, YouTube, or Spotify.
- Video conferencing: a technology that allows users to communicate with each other in real time using audio and video over the internet, such as Zoom, Skype, or Google Meet.
- Internet tools have various uses, advantages, and disadvantages, depending on the context, purpose, and user preferences. Some of the common uses are:
  - To access information and knowledge from various sources and domains.
  - To communicate and collaborate with other people across different locations and time zones.
  - To learn new skills and enhance existing ones.
  - To create and share content and express oneself creatively.
  - To perform tasks and activities more efficiently and conveniently.
  - To entertain oneself and enjoy various forms of media.
- Some of the common advantages are:
  - They are easy to use and accessible to anyone with an internet connection and a device.
  - They are cost-effective and often free or low-cost.
  - They are flexible and customizable to suit different needs and preferences.
  - They are interactive and engaging and allow users to provide and receive feedback.
  - They are scalable and adaptable to changing demands and situations.
- Some of the common disadvantages are:
  - They require a reliable and fast internet connection and a compatible

- device, which may not be available or affordable to everyone.
- They may pose security and privacy risks, such as data breaches, hacking, or identity theft.
  - They may cause distraction and addiction, as users may spend too much time or attention on them.
  - They may reduce human interaction and social skills, as users may rely more on virtual communication than face-to-face communication.
  - They may have negative effects on health and well-being, such as eye strain, headaches, or stress.

### **Introduction to client-server computing**

- Client-server computing is a software architecture model that divides an application into two parts: client systems and server systems .
- A client system is a program or device that requests a service or resource from a server system, which is a program or device that provides the service or resource .
- Client and server systems can communicate over a computer network or on the same computer .
- Client-server computing enables distributed processing of tasks, data sharing, scalability, security, and interoperability among different systems .
- Examples of client-server applications are web browsers and web servers, email clients and email servers, database clients and database servers, etc.

### **Core Java**

Core Java is the basic and fundamental part of the Java programming language that defines the syntax, data types, operators, control structures, classes, interfaces, exceptions, and input/output operations. It is also known as the Java Standard Edition (Java SE) and covers the core APIs that are used for general-purpose applications.

Some of the main features of Core Java are:

- It is platform-independent, meaning that it can run on any operating system that supports a Java Virtual Machine (JVM).
- It is object-oriented, meaning that it organizes data and behavior into reusable and modular units called objects.
- It is compiled and interpreted, meaning that it converts the source code into bytecode that can be executed by the JVM at runtime.
- It is strongly typed, meaning that it enforces strict rules for data types and conversions.
- It is multithreaded, meaning that it can create and manage multiple threads of execution that can run concurrently.
- It is robust, meaning that it has mechanisms for error handling and memory management that prevent crashes and memory leaks.

- It is secure, meaning that it has features for authentication, encryption, and access control that protect the data and code from unauthorized access.
- It is high-performance, meaning that it has optimizations for speed and efficiency that make it suitable for various applications.

Some of the topics that are covered in Core Java are:

- Variables and data types: These are the basic units of data that can store values of different types, such as int, double, char, boolean, etc.
- Operators and expressions: These are the symbols and rules that define how to manipulate and evaluate the data, such as +, -, \*, /, %, ==, !=, etc.
- Control structures: These are the statements that control the flow of execution based on certain conditions, such as if, else, switch, for, while, do-while, break, continue, etc.
- Arrays and strings: These are the data structures that can store multiple values of the same type in a sequential order, such as int[], char[], String, etc.
- Methods and parameters: These are the blocks of code that can perform a specific task and can be invoked with or without arguments, such as main(), println(), Math.sqrt(), etc.
- Classes and objects: These are the blueprints and instances of the object-oriented paradigm that define the state and behavior of the data, such as Person, Student, Employee, etc.
- Inheritance and polymorphism: These are the concepts that allow the reuse and modification of the existing code by creating new classes that inherit or override the features of the parent classes, such as Animal, Dog, Cat, etc.
- Interfaces and abstract classes: These are the contracts and templates that define the common behavior and structure of the classes that implement or extend them, such as Comparable, Runnable, Shape, etc.
- Exceptions and assertions: These are the events and statements that handle the abnormal situations and errors that may occur during the execution, such as try, catch, finally, throw, throws, assert, etc.
- Collections and generics: These are the frameworks and mechanisms that provide more advanced and flexible data structures and types, such as List, Set, Map, ArrayList, HashSet, HashMap, etc.
- Input/output and serialization: These are the processes and techniques that enable the reading and writing of data from and to various sources and formats, such as files, streams, sockets, etc.

## Introduction to Java

- Java is a **computer programming language and platform** that was first released in 1995 by Sun Microsystems .
- Java is a **class-based, object-oriented, and general-purpose lan-**

guage that is designed to have as few implementation dependencies as possible .

- Java is intended to let application developers **write once, run anywhere (WORA)**, meaning that compiled Java code can run on all platforms that support Java without the need for recompilation .
- Java is a **popular and widely used** language, with more than 3 billion devices running Java and millions of developers working with it .
- Java is used for various kinds of applications, such as **mobile applications (especially Android apps), desktop applications, web applications, web servers, and application servers** .
- Java is also known for its **high performance, security, portability, and scalability** features .
- Java has several **editions** that cater to different domains and platforms, such as Java SE (Standard Edition), Java EE (Enterprise Edition), Java ME (Micro Edition), and Java FX (Graphics and Media)<sup>3</sup>.
- Java follows a **syntax** that is similar to C and C++, but with some differences and simplifications .
- Java programs consist of **classes** that define the data and behavior of objects, and **methods** that perform actions on those objects .
- Java programs are compiled into **bytecode**, which is a platform-independent representation of the program that can be executed by a **Java Virtual Machine (JVM)** .
- Java also supports **libraries** and **frameworks** that provide reusable code and functionality for various tasks and domains . Some examples are Java Collections, Java IO, Java Swing, JavaFX, Spring, Hibernate, etc.
- Java is a **dynamic and evolving** language that has undergone many changes and improvements over the years, with new features and enhancements being added in every major release . The latest version of Java is Java 17, which was released in September 2021.

## Data type in Core Java

- A data type specifies the type of value a variable can store or the type of an expression.
- Java is a strongly typed language, which means that you should specify the type of a variable before using it in a Java program.
- Data types in Java are divided into two groups: primitive and non-primitive .
- Primitive data types are the basic types of data that are built-in to the language and are not derived from any class or object.
- There are eight primitive data types in Java: byte, short, int, long, char, float, double, and boolean .
- Each primitive data type has a fixed size and range of values.
- Non-primitive data types are derived from classes, interfaces, and arrays .
- Non-primitive data types are also called reference types because they refer to objects or arrays that are stored in the heap memory.

- Non-primitive data types can have unlimited size and values, depending on the implementation of the class or interface.
- Examples of non-primitive data types are String, Object, Integer, ArrayList, etc .

### Variable in Core Java

- A variable is a named memory location that can store a value of a specific data type.
- Variables are used to store data and manipulate it during the execution of a program.
- Variables have three main characteristics: name, type and value.
- The name of a variable is an identifier that follows the naming rules and conventions of Java.
- The type of a variable determines the range of values that it can store and the operations that can be performed on it.
- The value of a variable is the data that is stored in the memory location associated with the variable name.
- Variables can be classified into two categories: primitive and reference.
- Primitive variables store the basic data types such as int, char, boolean, etc.
- Reference variables store the addresses of objects or arrays that are created dynamically in the heap memory.
- Variables can also be categorized based on their scope and lifetime: local, instance and static.
- Local variables are declared inside a method or a block and are accessible only within that method or block. They are created when the method or block is entered and destroyed when it is exited.
- Instance variables are declared inside a class but outside any method or block and are associated with each object of the class. They are created when the object is instantiated and destroyed when the object is garbage collected.
- Static variables are declared inside a class but outside any method or block and are associated with the class itself. They are created when the class is loaded and destroyed when the class is unloaded. They are shared by all the objects of the class.

### Arrays in Core Java

- An array is a collection of elements of the same data type that are stored in contiguous memory locations.
- An array can be declared by specifying the data type followed by square brackets and the array name, for example: `int[] numbers;`
- An array can be initialized by assigning a list of values enclosed in curly braces, for example: `int[] numbers = {1, 2, 3, 4, 5};`
- An array can also be initialized by specifying the size of the array and

allocating memory for it, for example: `int[] numbers = new int[5];`

- The size of an array can be obtained by using the `length` property of the array, for example: `int size = numbers.length;`
- The elements of an array can be accessed by using the array name and the index of the element, for example: `int first = numbers[0];`
- The index of an array starts from 0 and goes up to the size of the array minus one, for example: `numbers[0]` is the first element and `numbers[4]` is the last element in the array above.
- An array can be iterated over by using a for loop or an enhanced for loop, for example:

```
// using a for loop
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

```
// using an enhanced for loop
for (int number : numbers) {
    System.out.println(number);
}
```

- An array can be passed as a parameter to a method by using the array name, for example: `public static void printArray(int[] array) { ... }`
- An array can be returned from a method by using the array name, for example: `public static int[] reverseArray(int[] array) { ... }`
- An array can be multidimensional, meaning that it can have more than one dimension, for example: `int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};`
- A multidimensional array can be declared, initialized, accessed, iterated over, passed and returned in a similar way as a one-dimensional array, but with an additional index for each dimension, for example: `int element = matrix[1][2];`
- A multidimensional array can have different lengths for each dimension, for example: `int[][] jagged = {{1, 2}, {3, 4, 5}, {6}};`
- An array can be sorted by using the `Arrays.sort()` method from the `java.util` package, for example: `Arrays.sort(numbers);`
- An array can be searched by using the `Arrays.binarySearch()` method from the `java.util` package, for example: `int index = Arrays.binarySearch(numbers, 4);`
- An array can be copied by using the `Arrays.copyOf()` or `Arrays.copyOfRange()` methods from the `java.util` package, for example: `int[] copy = Arrays.copyOf(numbers, 3);`
- An array can be compared by using the `Arrays.equals()` or `Arrays.deepEquals()` methods from the `java.util` package, for example: `boolean equal = Arrays.equals(numbers, copy);`
- An array can be converted to a string by using the `Arrays.toString()`

or `Arrays.deepToString()` methods from the `java.util` package, for example: `String str = Arrays.toString(numbers);`

## Methods & Classes in Core Java

- A method is a block of code that performs a specific task. A method can be invoked or called by another method, by an object, or by itself.
- A class is a blueprint or template for creating objects. A class defines the attributes and behaviors of the objects of that type.
- A class can have one or more methods that define the actions or operations that the objects of that class can perform.
- A method can have zero or more parameters that specify the input values for the method. A method can also have a return type that specifies the output value of the method.
- A method can be declared as static or non-static. A static method belongs to the class and can be called without creating an object of the class. A non-static method belongs to the object and can be called only by using an object of the class.
- A method can be declared as public, private, protected, or default. These are called access modifiers and they determine the visibility or accessibility of the method from other classes or methods.
- A method can be declared as final, abstract, or synchronized. These are called non-access modifiers and they provide additional information or functionality to the method.
- A method can be overloaded or overridden. Method overloading means defining multiple methods with the same name but different parameters. Method overriding means redefining a method in a subclass that was already defined in a superclass.
- A class can be declared as public, private, protected, or default. These are also access modifiers and they determine the visibility or accessibility of the class from other classes or packages.
- A class can be declared as final, abstract, or interface. These are also non-access modifiers and they provide additional information or functionality to the class.
- A class can be inherited or extended by another class. This means that the subclass inherits or acquires all the attributes and behaviors of the superclass, and can also add its own attributes and behaviors.
- A class can implement one or more interfaces. An interface is a collection of abstract methods that a class must implement. An interface defines a contract or a specification for the class that implements it.
- A class can have one or more constructors that are used to initialize the objects of the class. A constructor has the same name as the class and does not have a return type.
- A class can have one or more fields or variables that store the state or data of the objects of the class. A field can be declared as static or non-static, and as public, private, protected, or default. A field can also be declared



as `final`, `transient`, or `volatile`. These are also non-access modifiers and they provide additional information or functionality to the field.

### Inheritance in Core Java

- Inheritance is a mechanism that allows one class to acquire the properties and behaviors of another class.
- The class that inherits from another class is called the subclass or derived class.
- The class that is inherited by another class is called the superclass or base class.
- Inheritance enables code reuse, polymorphism, and abstraction.
- In Java, inheritance is achieved by using the **`extends`** keyword.
- A subclass can inherit from only one superclass in Java, but a superclass can have multiple subclasses.
- A subclass inherits all the public and protected members of its superclass, but not the private members.
- A subclass can access the inherited members directly, or override them to provide its own implementation.
- A subclass can also define new members that are not present in its superclass.
- A subclass can use the **`super`** keyword to refer to its immediate superclass, and invoke its constructors and methods.
- A subclass can also use the **`this`** keyword to refer to itself, and invoke its own constructors and methods.
- Java supports multiple levels of inheritance, where a subclass can inherit from another subclass, and so on.
- Java also supports a special type of inheritance called interface inheritance, where a class can implement one or more interfaces that declare abstract methods.
- A class that implements an interface must provide concrete implementations for all the abstract methods declared in the interface, or be declared as abstract itself.
- A class can implement multiple interfaces, but an interface can extend only one other interface.
- A class that implements an interface can also inherit from another class, and vice versa.
- A class that implements an interface can use the **`implements`** keyword, and an interface that extends another interface can use the **`extends`** keyword.

### Package and Interface in Core Java

A package is a group of classes, interfaces, and sub-packages that are related to each other. A package helps to organize the code, avoid naming conflicts, and reuse existing types. An interface is a group of abstract methods that define a contract or a behavior for a class that implements it. An interface helps to

achieve abstraction, polymorphism, and multiple inheritance in Java.

Some of the points to remember about packages and interfaces in core Java are:

- A package is created using the **package** keyword, followed by the package name. For example, **package com.example;**.
- An interface is created using the **interface** keyword, followed by the interface name. For example, **interface Shape;**.
- A package can contain classes, interfaces, and sub-packages, but not variables or methods. An interface can contain only abstract methods, constants, and nested types, but not constructors or instance variables.
- A package can be imported using the **import** keyword, followed by the package name or a specific type name. For example, **import java.util.\*;** or **import java.util.Scanner;**.
- An interface can be implemented by a class using the **implements** keyword, followed by the interface name. For example, **class Circle implements Shape;**.
- A package can have a hierarchical structure, where a dot (.) is used to separate the sub-packages. For example, **java.util** is a sub-package of **java**. An interface can have a single or multiple inheritance, where an interface can extend one or more interfaces using the **extends** keyword. For example, **interface Printable extends Shape, Colorable;**.
- A package can have different levels of access modifiers, such as public, protected, default, and private. A public package or type can be accessed from anywhere, a protected package or type can be accessed from the same package or a subclass, a default package or type can be accessed from the same package only, and a private package or type can be accessed from the same class only. An interface can have only public or default access modifiers, where a public interface can be accessed from anywhere, and a default interface can be accessed from the same package only.
- A package can have different types of annotations, such as **@Deprecated**, **@SafeVarargs**, **@FunctionalInterface**, etc. An annotation is a special kind of comment that provides additional information or metadata about the package or type. An interface can have only one type of annotation, which is **@FunctionalInterface**. A functional interface is an interface that has only one abstract method, and can be used as a lambda expression or a method reference.

## Exception Handling in Core Java

- An exception is an abnormal or unexpected event that occurs during the execution of a program and disrupts its normal flow.
- Exception handling is a mechanism that allows a program to deal with exceptions gracefully and prevent the program from terminating abruptly.
- Exception handling in core Java involves four keywords: try, catch, throw and finally.
- The try block contains the code that may cause an exception. The catch

block contains the code that handles the specific exception. The finally block contains the code that is always executed regardless of whether an exception occurs or not.

- The `throw` keyword is used to explicitly throw an exception from a method or a block of code. The `throws` keyword is used to declare that a method may throw one or more exceptions.
- Java provides two types of exceptions: checked and unchecked. Checked exceptions are those that are checked by the compiler at compile-time and must be handled or declared by the programmer. Unchecked exceptions are those that are not checked by the compiler and are usually caused by runtime errors or logical errors.
- Some of the common checked exceptions are `IOException`, `FileNotFoundException`, `ClassNotFoundException`, etc. Some of the common unchecked exceptions are `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException`, etc.
- Java also provides a hierarchy of exception classes that inherit from the `java.lang.Throwable` class. The `Throwable` class has two subclasses: `java.lang.Exception` and `java.lang.Error`. The `Exception` class is the superclass of all checked and unchecked exceptions. The `Error` class is the superclass of all errors that are usually fatal and should not be handled by the program.

## Multithread programming in Core Java

- Multithreading is a feature of Java that allows multiple threads of execution to run concurrently within a single program.
- A thread is a lightweight sub-process that can perform a specific task independently or in parallel with other threads.
- Threads share the same memory space and resources of the program, but have their own stack, program counter, and local variables.
- Threads can communicate with each other using shared variables, synchronization mechanisms, or inter-thread communication methods such as `wait`, `notify`, and `notifyAll`.
- Multithreading can improve the performance and responsiveness of a program by utilizing the available CPU cores and reducing the idle time of the processor.
- Multithreading can also create challenges such as concurrency issues, deadlock, starvation, race conditions, and memory consistency errors.
- To create and manage threads in Java, there are two main ways: extending the `Thread` class or implementing the `Runnable` interface.
- The `Thread` class provides methods to start, stop, suspend, resume, join, interrupt, and set the priority of a thread.
- The `Runnable` interface defines a single method `run()` that contains the logic of the thread.
- To start a thread, an instance of the `Thread` class or a class that implements the `Runnable` interface must be created and the `start()` method

must be invoked on it.

- The `start()` method creates a new thread of execution and calls the `run()` method of the corresponding object.
- The `run()` method defines the behavior of the thread and can be overridden by subclasses of the `Thread` class or classes that implement the `Runnable` interface.
- The main thread of a Java program is the one that executes the `main()` method and can create and control other threads.
- The main thread can wait for the completion of other threads by using the `join()` method of the `Thread` class.
- The main thread can also terminate the program by calling the `System.exit()` method or by returning from the `main()` method.

## I/O in Core Java

- I/O stands for input/output, which refers to the process of reading and writing data from and to various sources and destinations, such as files, network, console, etc.
- Core Java provides a rich set of classes and interfaces in the `java.io` package to handle different types of I/O operations, such as byte streams, character streams, buffered streams, data streams, object streams, file streams, etc.
- Byte streams are the lowest level of I/O abstraction, which operate on single bytes of data. They are useful for reading and writing binary data, such as images, audio, video, etc. The abstract classes `InputStream` and `OutputStream` are the base classes for all byte stream classes.
- Character streams are higher level of I/O abstraction, which operate on characters of data. They are useful for reading and writing text data, such as documents, source code, etc. The abstract classes `Reader` and `Writer` are the base classes for all character stream classes.
- Buffered streams are wrappers around other streams, which provide buffering functionality to improve the performance of I/O operations. They store data in an internal buffer and reduce the number of system calls to the underlying source or destination. The classes `BufferedInputStream`, `BufferedOutputStream`, `BufferedReader`, and `BufferedWriter` are examples of buffered streams.
- Data streams are wrappers around other streams, which provide methods to read and write primitive data types and strings in a platform-independent way. They are useful for storing and transferring binary data, such as numbers, booleans, etc. The classes `DataInputStream` and `DataOutputStream` are examples of data streams.
- Object streams are wrappers around other streams, which provide methods to read and write objects in a serialized form. They are useful for storing and transferring complex data structures, such as arrays, collections, etc. The classes `ObjectInputStream` and `ObjectOutputStream` are examples of object streams.

- File streams are streams that read and write data from and to files on the disk. They are subclasses of byte streams or character streams, depending on the type of data. The classes `FileInputStream`, `FileOutputStream`, `FileReader`, and `FileWriter` are examples of file streams.

### Java Applet in Core Java

- An applet is a small Java program that runs inside a web browser or an applet viewer.
- An applet can perform various tasks such as animation, calculation, games, etc.
- An applet has a limited access to the resources of the host system, such as files, network, etc. for security reasons.
- An applet follows a life cycle that consists of four stages: initialization, start, stop, and destroy.
- An applet can be embedded in an HTML page using the `<applet>` tag or the `<object>` tag.
- An applet can communicate with other applets running on the same page or different pages using the `AppletContext` class.
- An applet can also communicate with the web server using the `URL` and `URLConnection` classes.
- An applet can receive parameters from the HTML page using the `<param>` tag and the `getParameter()` method.
- An applet can override the methods of the `Applet` class or the `JApplet` class to perform various tasks.
- An applet can use the methods of the `Graphics` class or the `Graphics2D` class to draw graphics on the applet window.
- An applet can handle user events such as mouse clicks, keyboard inputs, etc. using the `Event` class or the `AWTEvent` class and the `EventListener` interface or the `AWTEventListener` interface.
- An applet can use the methods of the `AudioClip` interface or the `Clip` interface to play sound files on the applet window.
- An applet can use the methods of the `Image` class or the `BufferedImage` class to load and display images on the applet window.
- An applet can use the methods of the `Thread` class or the `Runnable` interface to create and run multiple threads on the applet window.
- An applet can use the methods of the `AppletStub` interface or the `AppletContext` interface to get information about the applet environment.

### String handling in Core Java

- String handling is a way of handling and manipulating strings in Java with the help of lot concepts like concatenation, comparison, etc.
- A string is a sequence of characters. In Java, a string is an object that is created by using the `String` class.

- The `String` class provides a lot of methods to perform operations on strings such as `compare()`, `concat()`, `equals()`, `split()`, `length()`, `replace()`, `compareTo()`, `intern()`, `substring()` etc.
- The `String` class implements `Serializable`, `Comparable` and `CharSequence` interfaces.
- There are two ways to create a string in Java:
  - String literal: `String s = "GeeksforGeeks";`
  - Using new keyword: `String s = new String("GeeksforGeeks");`
- String literals are stored in a special memory area called the string constant pool.
- Strings are immutable in Java, which means they cannot be changed once created.
- To create mutable strings, we can use classes like `StringBuffer` or `StringBuilder`.
- `StringBuffer` is a peer class of `String` that provides much of the functionality of strings. It is synchronized and thread-safe.
- `StringBuilder` is similar to `StringBuffer`, but it is not synchronized and not thread-safe. It is faster than `StringBuffer`.

## Event handling in Core Java

- An event is an action or occurrence that happens during the execution of a program, such as clicking a button, moving the mouse, typing a key, etc.
- Event handling is the process of responding to an event by performing some tasks, such as updating the user interface, validating the input, executing a command, etc.
- Event handling in Core Java involves the following components:
  - Event source: The object that generates an event, such as a button, a text field, a window, etc.
  - Event listener: The object that receives the notification of an event and defines the logic to handle it, such as a class that implements an interface or extends an abstract class.
  - Event object: The object that encapsulates the information about an event, such as its type, source, time, etc.
- Event handling in Core Java follows the delegation model, which means that the event source delegates the responsibility of handling the event to the event listener.
- To handle an event in Core Java, the following steps are required:
  - Register the event listener with the event source, using methods such as `addActionListener()`, `addMouseListener()`, `addWindowListener()`, etc.
  - Implement the event listener interface or extend the event adapter class, and override the methods that correspond to the events of interest, such as `actionPerformed()`, `mouseClicked()`, `windowClosing()`, etc.

- In the event listener methods, write the code to handle the event, using the event object as a parameter, and access its methods and fields, such as `getSource()`, `getType()`, `getX()`, `getY()`, etc.

## AWT Controls

- AWT stands for Abstract Window Toolkit, which is a set of APIs for creating graphical user interfaces (GUIs) in Java .
- AWT controls are components that allow a user to interact with the GUI in various ways, such as entering text, selecting options, clicking buttons, etc .
- AWT controls are also called heavyweight components, because they rely on the native operating system (OS) for their appearance and functionality .
- The `java.awt` package contains the classes and interfaces for AWT controls, such as `Label`, `Button`, `Checkbox`, `Choice`, `List`, `Scrollbar`, `TextField`, `TextArea`, etc .
- AWT controls can be added to containers, such as `Frame`, `Panel`, or `Dialog`, which are also components that can hold other components .
- AWT controls can be customized by setting their properties, such as size, location, color, font, text, etc .
- AWT controls can also handle events, such as mouse clicks, keyboard inputs, or window actions, by registering listeners or overriding methods .
- AWT controls are useful for creating simple and platform-dependent GUIs, but they have some limitations, such as lack of support for advanced features, inconsistent look and feel, and poor performance .

Some examples of AWT controls are:

- `Label`: A component for displaying text in a container.
- `Button`: A component for triggering an action when clicked.
- `Checkbox`: A component for selecting or deselecting an option.
- `Choice`: A component for displaying a drop-down list of options.
- `List`: A component for displaying a scrollable list of items.
- `Scrollbar`: A component for adjusting a value within a range.
- `TextField`: A component for entering a single line of text.
- `TextArea`: A component for entering multiple lines of text.

## Layout managers in AWT

- Layout managers are objects that implement the `LayoutManager` interface and determine the size and position of the components within a container.
- AWT provides several built-in layout managers, such as `FlowLayout`, `BorderLayout`, `GridLayout`, `CardLayout`, and `GridBagLayout`.
- Each container has a default layout manager that can be changed by calling the `setLayout` method with a new layout manager object as the argument.

- Layout managers can be nested within each other to create complex layouts.
- Some of the benefits of using layout managers are:
  - They simplify the code by avoiding the need to specify the exact coordinates and dimensions of each component.
  - They make the GUI adaptable to different screen resolutions, font sizes, and platform look-and-feels.
  - They allow the components to resize automatically when the container is resized or when new components are added or removed.

## Unit 2 - Web Page Designing

- Web page designing is the process of creating and arranging the content of a web page, such as text, images, links, videos, etc.
- Web page designing involves two aspects: web design and web development.
- Web design is the visual and aesthetic aspect of a web page, such as layout, color scheme, typography, etc. Web design is usually done using tools like HTML, CSS, and JavaScript.
- Web development is the functional and interactive aspect of a web page, such as navigation, user input, data processing, etc. Web development is usually done using tools like PHP, Python, Ruby, etc.
- Web page designing follows some principles and guidelines, such as usability, accessibility, responsiveness, consistency, etc.
- Web page designing also requires some skills and knowledge, such as creativity, communication, problem-solving, etc.

### List in Web Page Designing

- A list is a collection of items that are related in some way and displayed in a specific order.
- Lists are useful for organizing and presenting information in a clear and concise way.
- There are two types of lists in web page designing: ordered lists and unordered lists.
- An ordered list, also known as a numbered list, displays the items in a sequence using numbers, letters, or roman numerals as markers.
- An unordered list, also known as a bulleted list, displays the items in no particular order using symbols such as bullets, dashes, or squares as markers.
- To create a list in HTML, you need to use the `<ul>` tag for an unordered list, or the `<ol>` tag for an ordered list.
- Each item in the list is enclosed by the `<li>` tag, which stands for list item.
- You can nest one list inside another list by placing the `<ul>` or `<ol>` tag inside an `<li>` tag.



- You can also use the `<dl>` tag to create a definition list, which consists of pairs of terms and definitions.
- Each term is enclosed by the `<dt>` tag, and each definition is enclosed by the `<dd>` tag.
- You can style the lists using CSS properties such as `list-style-type`, `list-style-image`, `list-style-position`, and `list-style`.
- You can also use pseudo-elements such as `::marker` and `::before` to customize the appearance of the list markers.

## Images in Web Page Designing

- Images are visual elements that can enhance the appearance and functionality of a web page.
- Images can be used for various purposes, such as:
  - Illustrating the content or the theme of the web page.
  - Providing information or instructions to the users.
  - Creating visual interest or attraction for the users.
  - Establishing the identity or the brand of the web page or the website.
  - Linking to other web pages or external resources.
- Images can be of different types, such as:
  - Bitmap images, which are composed of pixels or dots of color. Examples of bitmap image formats are JPEG, PNG, GIF, BMP, etc.
  - Vector images, which are composed of mathematical shapes and curves. Examples of vector image formats are SVG, EPS, PDF, etc.
  - Animated images, which are sequences of images that create the illusion of movement. Examples of animated image formats are GIF, APNG, WebP, etc.
- Images can be embedded in a web page using different methods, such as:
  - The `<img>` tag, which is an inline element that displays an image specified by the `src` attribute. The `<img>` tag can also have other attributes, such as `alt`, `width`, `height`, `title`, etc.
  - The `background-image` property, which is a CSS property that sets an image as the background of an element. The `background-image` property can also have other properties, such as `background-repeat`, `background-position`, `background-size`, etc.
  - The `url()` function, which is a CSS function that inserts an image as a value of a property, such as `content`, `border-image`, `list-style-image`, etc.
- Images can be optimized for web page designing, such as:
  - Choosing the appropriate image format and quality for the intended purpose and audience of the web page.
  - Resizing and cropping the images to fit the layout and design of the web page.
  - Compressing and reducing the file size of the images to improve the loading speed and performance of the web page.
  - Adding alternative text and captions to the images to improve the

accessibility and SEO of the web page.

### Frames in Web Page Designing

- Frames are a feature of HTML that allows you, as the author, to control the layout of a Website in the user's browser.
- Frames allow you to divide the browser window into rectangular sections that can be treated as if they were separate browser windows. They can be scrolled and resized, and loaded with different Web pages.
- A frames page, also called a frameset, is a Web page that is divided into two or more sections, each of which points to another Web page. A frame on a frames page can also point to another frames page.
- To use frames on a page, we use `<frameset>` tag instead of `<body>` tag. The `<frameset>` tag defines how to divide the window into frames. The `rows` attribute of `<frameset>` tag defines horizontal frames and `cols` attribute defines vertical frames.
- A frame is defined by using `<frame>` tag inside the `<frameset>` tag. The `<frame>` tag has a `src` attribute that specifies the URL of the Web page to be displayed in the frame. The `<frame>` tag can also have other attributes, such as `name`, `scrolling`, `noresize`, `marginwidth`, `marginheight`, etc.
- Frames can be useful for creating consistent navigation menus, headers, footers, etc. However, frames also have some disadvantages, such as :
  - Frames can disrupt the flow of the Web and make it difficult to bookmark, link, or print specific pages .
  - Frames can cause accessibility problems for some users, such as those using screen readers, text browsers, or mobile devices .
  - Frames can increase the loading time and bandwidth usage of a Web page, as each frame requires a separate HTTP request .
  - Frames can create compatibility issues with some browsers or devices that do not support frames or have different screen resolutions .

### Forms in Web Page Designing

- Forms are web pages that allow users to enter and submit their information to a server or a database.
- Forms are composed of different types of input elements, such as text fields, checkboxes, radio buttons, sliders, buttons, etc.
- Forms can be used for various purposes, such as collecting user feedback, requesting web design services, registering for an account, logging in to a website, placing an order, booking a reservation, etc.
- Forms should be designed with the following best practices in mind :
  - Structure the form logically and clearly, using headings, labels, and groups to guide the user.

- Use appropriate input fields for the type of data you want to collect, and provide validation and error messages to help the user enter the correct information.
- Keep the form as short and simple as possible, and only ask for the essential information. Use progressive disclosure or multi-step forms to break down long or complex forms into manageable chunks.
- Make the form visually appealing and consistent with the overall website design, using colors, fonts, icons, and images to enhance the user experience.
- Optimize the form for mobile devices, using responsive design, touch-friendly inputs, and clear buttons. Avoid horizontal scrolling and pop-ups that may interfere with the user's input.
- Test the form for usability and functionality, and measure the conversion rate and user feedback to identify and improve any issues.

## CSS in Web Page Designing

CSS stands for Cascading Style Sheets. It is a language that is used to style and layout web pages, by applying rules to HTML elements. Some of the benefits of using CSS are:

- It separates the presentation from the content, making the web pages easier to maintain and update.
- It allows for consistent styling across multiple web pages, by using external style sheets that can be linked or imported.
- It gives more control over the appearance and behavior of the web pages, by using properties such as color, font, size, margin, padding, border, display, position, and animation.
- It enables responsive web design, by using media queries and flexible units that can adapt to different screen sizes and devices.

To use CSS in web page designing, there are three ways to insert CSS:

- **Inline CSS:** The `style` attribute is used to apply CSS rules to a single HTML element. This method is not recommended, as it mixes the presentation with the content and creates redundancy.
- **Internal CSS:** The `<style>` element is used to define CSS rules in the `<head>` section of an HTML document. This method is useful for applying CSS to a single web page, but it still creates some redundancy if the same style is used in multiple web pages.
- **External CSS:** The `<link>` element is used to link an external CSS file to an HTML document. This method is the most efficient and recommended way to use CSS, as it allows for reusing the same style sheet in multiple web pages and reduces the file size and loading time of the web pages.

To apply CSS rules to HTML elements, there are two main concepts to understand:

- **Selectors:** These are patterns that match one or more HTML elements, based on their attributes, classes, ids, or relationships. For example, `p` is a selector that matches all `<p>` elements, `.red` is a selector that matches all elements with the class attribute `red`, and `#logo` is a selector that matches the element with the id attribute `logo`.
- **Declarations:** These are statements that define the style properties and values for the selected elements. For example, `color: blue;` is a declaration that sets the text color to blue. A declaration consists of a property and a value, separated by a colon and ending with a semicolon. A declaration block is a group of declarations enclosed by curly braces. A rule set is a combination of a selector and a declaration block.

Here is an example of a CSS rule set that styles all `<h1>` elements with a red text color and a blue background color:

```
h1 {
  color: red;
  background-color: blue;
}
```

To create a web page layout with CSS, there are several techniques and properties that can be used, such as:

- The **display** property: This property defines how an element is displayed on the web page, such as `block`, `inline`, `inline-block`, `flex`, `grid`, or `none`. Block elements take up the full width of the web page and start on a new line, while inline elements take up only the space they need and can be on the same line as other elements. Inline-block elements are a combination of block and inline elements, as they can have a width and height, but can also be on the same line as other elements. Flex and grid elements are used to create flexible and responsive layouts, by arranging their child elements in rows or columns, with alignment and spacing options. None elements are hidden from the web page and do not take up any space.
- The **position** property: This property defines how an element is positioned on the web page, relative to its normal position, its parent element, or the viewport. There are five values for this property: `static`, `relative`, `absolute`, `fixed`, and `sticky`. Static elements are positioned according to their normal flow, and are the default value for this property. Relative elements are positioned relative to their normal position, by using the `top`, `right`, `bottom`, and `left` properties to offset them. Absolute elements are positioned relative to their nearest positioned ancestor, or the document body if none exists, by using the `top`, `right`, `bottom`, and `left` properties to place them. Fixed elements are positioned relative to the viewport, and do not move when the web page is scrolled. Sticky elements are positioned relative to their normal position, until they reach a specified threshold, and then they become fixed.
- The **float** and **clear** properties: These properties are used to create layouts where elements are placed side by side, or wrap around each other.

The

### Document type definition in Web Page Designing

- A document type definition (DTD) is an instruction that tells the web browser about the markup language in which the current page is written.
- A DTD defines the structure and the legal elements and attributes of an XML document.
- A DTD can be declared inside an XML document as internal or as an external reference.
- A DTD helps to ensure that the web page is parsed the same way by different web browsers and that the XML data is valid.
- A DTD can also be used to agree on a standard format for interchanging data between independent groups of people.
- A DTD is not mandatory for HTML documents, but it is recommended to use one to avoid rendering mode differences.
- A DTD is part of the Document interface, which represents any web page loaded in the browser and serves as an entry point into the web page's content, which is the DOM tree.

### XML in Web Page Designing

- XML stands for **Extensible Markup Language**. It is a markup language that uses tags to define data and information on the web .
- XML is similar to HTML, but with some key differences. One of the main advantages of using XML to design web pages is that it allows for more **flexibility** in the design. You can create your own tags and attributes to suit your needs, and you can also use XML to separate the data from the presentation.
- XML is also useful for **storing and transporting** data across different platforms and applications. XML data can be easily parsed and manipulated by various tools and languages, such as JavaScript, PHP, Java, etc. XML data can also be validated and transformed using schemas and stylesheets.
- XML is not a replacement for HTML, but rather a **complement** to it. XML can be used to store the data and information that is displayed on the web page, while HTML can be used to define the structure and layout of the web page. XML can also be embedded in HTML documents using the `<script>` or `<object>` tags.
- XML is a powerful and flexible way to design web pages that can store, search, and share data and information on the web. XML is widely used in web development and web services, such as RSS feeds, SOAP messages, REST APIs, etc .

### DTD in Web Page Designing

- DTD stands for Document Type Definition.

- A DTD defines the structure and the legal elements and attributes of an XML document .
- A DTD can be declared inside an XML document as inline or as an external recommendation.
- A DTD helps to verify that XML data is valid and conforms to the rules of a DTD .
- A DTD also helps independent groups of people to agree on a standard DTD for interchanging data.
- A DTD affects the display of the web page by defining the type of document and the specific version of the HTML spec you intended to target.
- A DTD can be used for HTML-, XHTML-, or HTML5-based pages.
- A DTD consists of two parts: definition and declaration.
- The definition matches the DTD type to the type of code used in the document.
- The declaration specifies the URL of a web-accessible document that contains the DTD rules.
- An example of a DTD declaration for an HTML5 document is:

```
<!DOCTYPE html>
```

- An example of a DTD declaration for an XML document that uses an external DTD file is:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
</note>
```

## XML Schemes in Web Page Designing

- XML stands for eXtensible Markup Language. It is a markup language similar to HTML, but without predefined tags to use. Instead, you define your own tags designed specifically for your needs. This is a powerful way to store and transport data in a format that can be stored, searched, and shared.
- XML schemas are used to define the structure, content, and constraints of XML documents. They are written in XML syntax and can be validated by XML processors. XML schemas are an alternative to DTDs (Document Type Definitions), which are less expressive and less flexible.
- XML schemas support data types, which allow you to specify the format and range of values for the elements and attributes in your XML documents. Data types can be built-in (such as string, integer, date, etc.) or user-defined (by deriving from existing types or creating lists or unions of types). Data types make it easier to describe, validate, and restrict your data.

- XML schemas use various design patterns to organize the global elements and types in a schema. Some of the common design patterns are:
  - Russian Doll: All the elements and types are nested inside the root element, which is the only global element. This pattern is simple and compact, but it makes reuse and extension difficult.
  - Salami Slice: All the elements are global and have anonymous types. This pattern is easy to reuse and extend, but it makes the schema verbose and hard to read.
  - Venetian Blind: All the types are global and have names, and the elements have references to the types. This pattern is modular and flexible, but it requires more indirection and navigation.
  - Garden of Eden: All the elements and types are global and have names. This pattern is the most expressive and reusable, but it may introduce name conflicts and redundancy.
- XML schemas can be built in-memory using the classes in the System.Xml.Schema namespace, which map to the structures defined in the W3C XML Schema Recommendation. The XmlSchema class represents the root of a schema, and the XmlSchemaElement and XmlSchemaType classes represent the elements and types in a schema. You can use the XmlSchemaSet class to compile and validate a schema.

## Object Models in Web Page Designing

- An object model is a representation of the components and relationships of a web page or a web application.
- An object model can help to organize the structure, behavior, and presentation of a web page or a web application.
- An object model can also facilitate the reuse and maintenance of the web page or web application code.
- There are different types of object models, such as document object model (DOM), browser object model (BOM), and component object model (COM).
- The document object model (DOM) is a standard interface that defines how HTML and XML documents are represented and manipulated by web browsers and scripts.
- The browser object model (BOM) is a collection of objects that provide access to the browser's features and settings, such as window, location, history, navigator, screen, and document.
- The component object model (COM) is a technology that allows software components to communicate and interact with each other across different applications and platforms.
- An object model can be represented by a diagram that shows the hierarchy, properties, methods, and events of the objects. For example, the following diagram shows a simplified version of the DOM for an HTML document:

DOM diagram

- An object model can be accessed and manipulated by using scripting languages, such as JavaScript, that can create, modify, delete, and traverse the objects and their attributes. For example, the following code snippet shows how to access and change the text content of an HTML element using the DOM:

```
// Get the element with id="demo"
var element = document.getElementById("demo");

// Change the text content of the element
element.textContent = "Hello, world!";
```

## Presenting and using XML in Web Page Designing

XML stands for Extensible Markup Language. It is a language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. XML is widely used for storing and exchanging data on the web.

Some of the benefits of using XML in web page designing are:

- XML is platform-independent and can be used with any programming language or software.
- XML is self-describing and can define the structure and meaning of the data.
- XML is extensible and can be customized to suit the needs of different applications and domains.
- XML can be validated against a schema or a document type definition (DTD) to ensure the data is well-formed and conforms to the rules.
- XML can be transformed into other formats using XSLT (Extensible Stylesheet Language Transformations) or other tools.

Some of the ways of presenting and using XML in web page designing are:

- XML can be embedded in HTML using the `<script>` tag or the `<object>` tag. This allows the XML data to be accessed and manipulated by JavaScript or other scripts in the web page.
- XML can be linked to HTML using the `<link>` tag or the `<a>` tag. This allows the XML data to be retrieved and displayed by the web browser or other applications.
- XML can be styled using CSS (Cascading Style Sheets) or XSL (Extensible Stylesheet Language). This allows the XML data to be formatted and presented in a consistent and attractive way.
- XML can be queried using XPath (XML Path Language) or XQuery (XML Query Language). This allows the XML data to be searched and filtered based on certain criteria or conditions.
- XML can be processed using DOM (Document Object Model) or SAX (Simple API for XML). This allows the XML data to be accessed and



manipulated as a tree or a stream of events.

### Using XML Processors in Web Page Designing

- XML processors are software applications that parse, validate, and manipulate XML documents.
- XML processors can be classified into two types: validating and non-validating.
- Validating XML processors check the syntax and structure of XML documents against a schema or a DTD (Document Type Definition), and report any errors or warnings.
- Non-validating XML processors only check the syntax of XML documents, and do not perform any validation.
- XML processors can be used in web page designing for various purposes, such as:
  - Transforming XML documents into HTML or other formats using XSLT (Extensible Stylesheet Language Transformations).
  - Extracting data from XML documents using XPath (XML Path Language) or XQuery (XML Query Language).
  - Creating dynamic web pages using XML and scripting languages, such as JavaScript or PHP.
  - Validating user input or web forms using XML Schema or DTD.
  - Storing and retrieving data from XML databases or web services.

### DOM and SAX in Web Page Designing

- DOM and SAX are two different ways of processing XML documents in web page designing.
- DOM stands for Document Object Model, which is a standard for representing and manipulating XML documents as a tree of nodes. DOM allows reading and writing of XML documents, as well as querying and modifying them using methods and properties .
- SAX stands for Simple API for XML, which is an event-driven interface for parsing XML documents. SAX does not create a tree representation of the document, but instead notifies the application of the occurrence of different elements, attributes, text, etc. SAX allows only reading of XML documents, not writing .
- The main differences between DOM and SAX are :
  - DOM requires enough memory to store the whole document, while SAX processes it piece by piece. For very large XML documents, DOM may not be feasible, while SAX still works.
  - DOM allows random access and manipulation of any part of the document, while SAX only allows sequential access and does not support modification.
  - DOM is easier to use and understand, while SAX is more complex and requires more coding.

- SAX is faster and more efficient than DOM, as it does not create a tree structure and does not consume memory. DOM is slower and more memory-intensive, as it creates and maintains a tree structure.

## Dynamic HTML in Web Page Designing

- Dynamic HTML (DHTML) is a term that refers to the combination of HTML, CSS, JavaScript, and the Document Object Model (DOM) to create interactive and dynamic web pages.
- DHTML allows the web page to respond to user actions without reloading the page from the server. For example, DHTML can be used to create animations, menus, forms, pop-ups, drag-and-drop features, and more.
- DHTML uses the following components:
  - HTML: The markup language that defines the structure and content of the web page.
  - CSS: The style sheet language that defines the presentation and layout of the web page.
  - JavaScript: The scripting language that enables the manipulation and interaction of the web page elements.
  - DOM: The interface that represents the web page as a tree of objects that can be accessed and modified by JavaScript.
- DHTML works by using JavaScript to dynamically change the properties and attributes of the HTML elements and CSS styles, as well as to create and delete elements and styles. The DOM provides the methods and properties for JavaScript to access and modify the web page elements.
- DHTML has many advantages, such as:
  - Improving the user experience and engagement by providing interactivity and feedback.
  - Reducing the server load and bandwidth consumption by minimizing the need for page refreshes.
  - Enhancing the functionality and usability of the web page by adding features that are not possible with static HTML.
  - Increasing the accessibility and compatibility of the web page by allowing the adaptation to different devices, browsers, and user preferences.
- DHTML also has some disadvantages, such as:
  - Increasing the complexity and maintenance of the web page code by requiring more skills and tools.
  - Creating potential security and privacy issues by exposing the web page code and data to the user and third parties.
  - Causing compatibility and performance issues by relying on the browser support and execution of JavaScript and CSS.
  - Reducing the search engine optimization (SEO) and indexing of the web page by making the content less visible and crawlable.

## Unit 3 - Scripting

- Scripting is the process of writing and executing a series of commands or instructions that can be run by a computer program or system.
- Scripting languages are high-level languages that are interpreted or compiled at run time, rather than compiled in advance.
- Scripting languages are often used for automation, data processing, web development, testing, and other tasks that require dynamic and flexible behavior.
- Some examples of scripting languages are Python, Ruby, Perl, JavaScript, Bash, and PowerShell.
- Scripting languages have some advantages and disadvantages compared to compiled languages, such as:
  - Advantages:
    - \* They are easy to learn and use, as they have simple syntax and features.
    - \* They are portable and platform-independent, as they can run on different operating systems and environments.
    - \* They are expressive and concise, as they can perform complex operations with fewer lines of code.
    - \* They are flexible and adaptable, as they can interact with other languages and programs, and modify themselves at run time.
  - Disadvantages:
    - \* They are slower and less efficient, as they have more overhead and less optimization than compiled languages.
    - \* They are less secure and reliable, as they have less error checking and debugging tools than compiled languages.
    - \* They are less standardized and consistent, as they have more variations and dialects than compiled languages.
- To write and run a script, one needs a text editor and a scripting engine or interpreter that can execute the script.
- A script can be executed in different ways, such as:
  - By typing the script name or path in a command-line interface or terminal.
  - By using a graphical user interface or a web browser that can run the script.
  - By embedding the script in another program or document that can invoke the script.
  - By using a scheduler or a trigger that can run the script at a specified time or event.

### JavaScript in Scripting

- JavaScript is a scripting language that can run in web browsers and other environments.
- Scripting languages are high-level languages that can be interpreted or

compiled at runtime, without the need for a separate compilation step.

- Scripting languages are often used for tasks such as web development, automation, data analysis, and testing.
- JavaScript has the following features as a scripting language:
  - It is dynamic, meaning that variables do not have fixed types and values can be changed at any time.
  - It is weakly typed, meaning that type conversions are done automatically and sometimes unpredictably.
  - It is prototype-based, meaning that objects inherit properties and methods from other objects, rather than from classes.
  - It is event-driven, meaning that code execution is triggered by events such as user input, network requests, or timers.
  - It is functional, meaning that functions are first-class objects that can be passed as arguments, returned as values, or stored in variables.
  - It is asynchronous, meaning that code execution can be deferred or interrupted by other tasks, such as promises, callbacks, or generators.
  - It supports multiple paradigms, such as imperative, declarative, object-oriented, and functional programming.
- JavaScript can be used for scripting in various contexts, such as:
  - Web browsers, where it can manipulate the Document Object Model (DOM), handle user interactions, communicate with servers, and create dynamic web pages.
  - Node.js, where it can run on the server-side and access the file system, databases, and network protocols.
  - Electron, where it can create cross-platform desktop applications using web technologies.
  - React Native, where it can create native mobile applications using web technologies.
  - TensorFlow.js, where it can perform machine learning and data science tasks using web technologies.

## Introduction to JavaScript

- JavaScript is a scripting language that runs in web browsers and enables dynamic and interactive web pages.
- JavaScript was created by Brendan Eich in 1995 and is standardized by the ECMAScript specification.
- JavaScript can manipulate HTML elements, handle user events, perform calculations, validate forms, communicate with servers, and create animations and games.
- JavaScript is case-sensitive, loosely typed, and supports object-oriented, functional, and imperative programming paradigms.

- JavaScript code can be embedded in HTML using the `<script>` tag or linked from an external file using the `src` attribute.
- JavaScript code can be executed by the browser when the page loads, when an event occurs, or when a function is called.
- JavaScript variables can store different types of values, such as numbers, strings, booleans, arrays, objects, functions, and symbols.
- JavaScript operators can perform arithmetic, assignment, comparison, logical, bitwise, and ternary operations on values.
- JavaScript statements can control the flow of execution, such as `if`, `else`, `switch`, `for`, `while`, `do-while`, `break`, `continue`, and `return`.
- JavaScript functions can be defined using function declarations, function expressions, arrow functions, or generator functions, and can be nested, hoisted, and invoked with different arguments and contexts.
- JavaScript objects can be created using object literals, constructors, or the `Object.create()` method, and can have properties and methods that can be accessed using dot or bracket notation.
- JavaScript arrays can be created using array literals, the `Array()` constructor, or the `Array.of()` and `Array.from()` methods, and can have elements of different types that can be accessed using indexes or iterators.
- JavaScript has several built-in objects, such as `Math`, `Date`, `String`, `Number`, `Boolean`, `RegExp`, `JSON`, `Map`, `Set`, `WeakMap`, `WeakSet`, `Promise`, `Proxy`, `Reflect`, and `Symbol`, that provide useful methods and properties for common tasks.
- JavaScript also has several built-in global functions, such as `eval()`, `parseInt()`, `parseFloat()`, `isNaN()`, `isFinite()`, `encodeURIComponent()`, `decodeURI()`, `encodeURIComponent()`, `decodeURIComponent()`, and `escape()`, that perform various operations on values.
- JavaScript supports error handling using the `try`, `catch`, `finally`, and `throw` statements, and has several built-in error objects, such as `Error`, `TypeError`, `SyntaxError`, `ReferenceError`, `RangeError`, and `URIError`.
- JavaScript supports asynchronous programming using callbacks, promises, `async/await`, and event-driven models, and can use the `XMLHttpRequest` or the `fetch` API to communicate with servers using HTTP requests and responses.
- JavaScript can be extended using various libraries and frameworks, such as `jQuery`, `React`, `Angular`, `Vue`, `Node.js`, `Express`, and many more, that provide additional features and functionalities for web development.

## Documents in JavaScript

- A document in JavaScript is an object that represents a web page loaded in a browser and provides access to its content, such as HTML elements, images, scripts, etc.
- A document is an instance of the `Document` interface, which is part of the Document Object Model (DOM), a programming interface that describes the structure and behavior of documents.

- The document object has many properties and methods that can be used to manipulate the web page, such as `document.title`, `document.body`, `document.getElementById()`, `document.createElement()`, `document.write()`, etc.
- The document object is a global object that can be accessed from any JavaScript code within the web page, using the keyword `document` or the `window.document` property.
- The document object can also be used to create new documents, such as XML or HTML documents, using the `document.implementation.createDocument()` or `document.implementation.createHTMLDocument()` methods.

### Forms in JavaScript

- Forms are an essential part of HTML pages that allow users to enter and submit data to a server.
- To create a form in HTML, you use the `<form>` element with attributes such as `action`, `method`, and `id`.
- To reference the `<form>` element in JavaScript, you can use DOM selecting methods such as `getElementById()`, `querySelector()`, or `forms` collection.
- To access the form fields (such as `input`, `textarea`, `select`, etc.), you can use the `elements` property of the form object, which returns a collection of the form controls.
- To get or set the value of a form field, you can use the `value` property of the element object.
- To validate the form data, you can use the `checkValidity()` method of the form object, which returns `true` or `false` depending on whether the form satisfies the constraints defined by the HTML attributes (such as `required`, `pattern`, `min`, `max`, etc.).
- To submit the form, you can use the `submit()` method of the form object, which sends the form data to the server specified by the `action` attribute. You can also use the `submit` event, which fires when the user clicks a submit button or presses the Enter key.
- To prevent the default behavior of the form submission, you can use the `preventDefault()` method of the event object, which allows you to perform custom actions before sending the form data to the server.

### Statements in JavaScript

- Statements are programming instructions written in a program .
- Statements are composed of values, expressions, operators, comments, and keywords.
- Statements are separated by semicolons (;) in JavaScript.
- Statements control the flow of a program and specify what scripts will do and how they will do it.
- There are different types of statements in JavaScript, such as conditional

statements, loop statements, function declarations, variable declarations, etc .

- Conditional statements are used to perform different actions based on different conditions, such as if, else, switch, etc.
- Loop statements are used to execute a block of code repeatedly until a certain condition is met, such as for, while, do...while, etc.
- Function declarations are used to define a function that can be called later in the program, such as function, return, etc.
- Variable declarations are used to declare and initialize variables that can store data, such as var, let, const, etc.

## Functions in JavaScript

- A function is a block of code that performs a specific task and can be reused in different parts of a program.
- A function can have zero or more parameters that are passed as values or references when the function is called.
- A function can return a value to the caller using the **return** statement, or it can return nothing (undefined) by default.
- A function can be defined in three ways: as a function declaration, as a function expression, or as an arrow function.
- A function declaration is a statement that starts with the **function** keyword, followed by the function name, the parameter list, and the function body. For example:

```
function add(a, b) {  
  return a + b;  
}
```

- A function expression is an expression that assigns a function to a variable or a property of an object. The function name is optional and can be omitted. For example:

```
var add = function(a, b) {  
  return a + b;  
};
```

```
var obj = {  
  add: function(a, b) {  
    return a + b;  
  }  
};
```

- An arrow function is a concise syntax for writing function expressions that uses the **=>** operator. It does not have its own **this**, **arguments**, **super**, or **new.target** keywords and inherits them from the enclosing scope. For example:

```
var add = (a, b) => a + b;
```

```
var obj = {  
  add: (a, b) => a + b  
};
```

- A function can be called by using the function name followed by the argument list in parentheses. For example:

```
var result = add(2, 3); // result is 5
```

- A function can also be called as a method of an object by using the dot notation or the bracket notation. For example:

```
var result = obj.add(2, 3); // result is 5  
var result = obj["add"](2, 3); // result is 5
```

- A function can be passed as an argument to another function or returned as a value from another function. This is known as a higher-order function. For example:

```
function multiplyByTwo(f) {  
  return function(x) {  
    return f(x) * 2;  
  };  
}
```

```
var doubleAdd = multiplyByTwo(add); // doubleAdd is a function  
var result = doubleAdd(2, 3); // result is 10
```

- A function can be nested inside another function and have access to the variables and parameters of the outer function. This is known as a closure. For example:

```
function makeCounter() {  
  var count = 0;  
  return function() {  
    return ++count;  
  };  
}
```

```
var counter = makeCounter(); // counter is a function  
var result = counter(); // result is 1  
var result = counter(); // result is 2
```

## Objects in JavaScript

- An object is a collection of properties that store values and functions that perform actions.



- An object can be created using an object literal, which is a pair of curly braces that enclose a list of property names and values, separated by commas.
- For example, `var person = {name: "Alice", age: 25, greet: function() {console.log("Hello, " + this.name);}};` creates an object named `person` with three properties: `name`, `age`, and `greet`.
- An object can also be created using an object constructor, which is a function that defines the properties and methods of the object.
- For example, `function Person(name, age) {this.name = name; this.age = age; this.greet = function() {console.log("Hello, " + this.name);}};` defines a constructor function named `Person` that takes two parameters: `name` and `age`.
- To create an object using a constructor, the `new` keyword is used, followed by the name of the constructor and the arguments.
- For example, `var person = new Person("Alice", 25);` creates an object named `person` using the `Person` constructor with the arguments "Alice" and 25.
- An object can be accessed and modified using dot notation or bracket notation.
- Dot notation uses a dot (`.`) followed by the property name to access or assign a value to a property.
- For example, `person.name` returns "Alice" and `person.name = "Bob"` changes the value of the `name` property to "Bob".
- Bracket notation uses square brackets (`[]`) with the property name as a string to access or assign a value to a property.
- For example, `person["name"]` returns "Alice" and `person["name"] = "Bob"` changes the value of the `name` property to "Bob".
- Bracket notation is useful when the property name is stored in a variable or contains special characters that are not allowed in dot notation.
- For example, `var prop = "name"; person[prop]` returns "Alice" and `person["first name"] = "Alice"` assigns a value to a property named "first name".
- An object can have methods, which are functions that are stored as properties of the object and can access and manipulate the object's data using the `this` keyword.
- For example, `person.greet()` calls the `greet` method of the `person` object, which logs "Hello, Alice" to the console.
- The `this` keyword refers to the current object that the method belongs to, and can be used to access and modify the object's properties.
- For example, `this.name` inside the `greet` method refers to the `name` property of the `person` object.

## Introduction to AJAX in JavaScript

- AJAX stands for Asynchronous JavaScript and XML. It is a technique for creating dynamic web pages that can update parts of the page without

reloading the whole page.

- AJAX uses the XMLHttpRequest object to send and receive data from a server in the background, without interfering with the current page.
- AJAX allows web pages to be more interactive and responsive, as the user does not have to wait for the page to refresh after every action.
- AJAX can use various data formats, such as XML, JSON, HTML, or plain text, to exchange data between the client and the server.
- AJAX can be implemented using various JavaScript libraries, such as jQuery, Angular, React, or Vue, that provide methods and functions to simplify the AJAX process.
- AJAX follows a basic workflow of:
  - Creating an XMLHttpRequest object
  - Opening a connection to the server
  - Sending a request to the server
  - Receiving a response from the server
  - Processing the response and updating the page
- An example of a simple AJAX request using plain JavaScript is:

```
// Create an XMLHttpRequest object
var xhr = new XMLHttpRequest();

// Open a connection to the server
xhr.open("GET", "data.txt", true);

// Send the request to the server
xhr.send();

// Define a function to handle the response
xhr.onreadystatechange = function() {
    // Check if the request is complete and successful
    if (xhr.readyState == 4 && xhr.status == 200) {
        // Get the response text
        var data = xhr.responseText;
        // Display the response text in an element with id="demo"
        document.getElementById("demo").innerHTML = data;
    }
};
```

## Networking in Scripting

- Networking is the process of connecting and communicating between different devices or systems over a network, such as the internet or a local area network (LAN).

- Scripting is the use of a programming language or a tool to automate tasks or manipulate data, such as Python, PowerShell, Bash, etc.
- Networking in scripting is the combination of both concepts, where scripts are used to perform network-related tasks or operations, such as sending or receiving data, scanning ports, testing connectivity, etc.
- Some examples of networking in scripting are:
  - Using the `socket` module in Python to create, bind, listen, accept, send and receive data over TCP or UDP sockets.
  - Using the `Invoke-WebRequest` cmdlet in PowerShell to send HTTP requests and parse the response from a web server or an API.
  - Using the `curl` command in Bash to transfer data from or to a server using various protocols, such as HTTP, FTP, SMTP, etc.
  - Using the `nmap` tool in Bash to scan ports, discover hosts, identify services, and perform security audits on a network.
- Some benefits of networking in scripting are:
  - It can automate repetitive or complex network tasks, saving time and effort.
  - It can enhance the functionality and performance of network applications or services, such as web servers, databases, etc.
  - It can facilitate the integration and interoperability of different network devices or systems, such as routers, switches, firewalls, etc.
  - It can enable the testing and debugging of network issues or vulnerabilities, improving the security and reliability of the network.

### Internet Addressing in Networking

- Internet addressing is the process of assigning unique identifiers to devices or interfaces that communicate over the internet using the Internet Protocol (IP) .
- Internet addresses are composed of two parts: a network address and a host address. The network address identifies the network or subnet to which the device belongs, and the host address identifies the specific device within that network or subnet .
- Internet addresses are usually written in dotted decimal notation, which consists of four decimal numbers separated by dots, such as 192.168.1.1. Each decimal number represents one byte (8 bits) of the 32-bit IP address .
- Internet addresses can be classified into different classes based on the number of bits allocated for the network and host parts. The classes are A, B, C, D, and E. Class A addresses have the first bit as 0, class B addresses have the first two bits as 10, class C addresses have the first three bits as 110, class D addresses have the first four bits as 1110, and class E addresses have the first four bits as 1111 .

- Class A, B, and C addresses are used for unicast communication, which means sending data to a single destination. Class D addresses are used for multicast communication, which means sending data to a group of destinations. Class E addresses are reserved for future use or experimental purposes .
- Internet addresses can also be divided into public and private addresses. Public addresses are globally unique and can be used to communicate with any device on the internet. Private addresses are not globally unique and can only be used within a private network, such as a home or office network. Private addresses need to be translated to public addresses using a technique called Network Address Translation (NAT) when communicating with devices outside the private network .
- Internet addresses can also be assigned dynamically or statically. Dynamic addressing means that a device obtains an internet address from a server or a pool of available addresses when it connects to the network. Static addressing means that a device has a fixed internet address that does not change. Dynamic addressing is more flexible and efficient, but static addressing is more reliable and secure .
- Internet addressing is essential for enabling data transmission and routing over the internet. It allows devices to identify and locate each other, and to exchange information using various protocols, such as TCP/IP . Internet addressing also enables network management and security functions, such as firewall and access control .

### **InetAddress in Networking**

- An InetAddress is an object that represents an Internet Protocol (IP) address, which is a numerical identifier for a device on a network.
- An InetAddress can be either an IPv4 address (32 bits) or an IPv6 address (128 bits).
- An InetAddress can also have a host name, which is a human-readable name for a device, such as www.example.com.
- An InetAddress can be created from a byte array, a host name, or a string representation of an IP address, using the static methods of the InetAddress class.
- An InetAddress can be used to communicate with other devices on the network, such as sending and receiving data, or performing network operations, such as ping or traceroute.
- An InetAddress can also provide information about the device, such as its local or loopback address, its canonical host name, its reachability, or its address type (IPv4 or IPv6).

### **Factory Methods in Networking**

- Factory methods are a design pattern that allows creating objects without specifying the exact class or constructor.

- Factory methods are useful for networking because they can abstract away the details of creating and configuring network connections, sockets, protocols, etc.
- Factory methods can also provide a consistent interface for creating different types of network objects, such as TCP, UDP, HTTP, etc.
- Factory methods can be implemented in different ways, such as:
  - Using a static method that takes a parameter to determine the type of object to create, e.g. `NetworkConnection.create("tcp")`.
  - Using a separate factory class that has methods for creating different types of objects, e.g. `NetworkConnectionFactory.createTcpConnection()`.
  - Using an abstract factory class that defines a common interface for creating network objects, and subclasses that implement the interface for specific types, e.g. `AbstractNetworkConnectionFactory` and `TcpNetworkConnectionFactory`.
- Factory methods can have advantages and disadvantages, such as:
  - Advantages:
    - \* They can reduce coupling between the client code and the network classes, making the code more flexible and maintainable.
    - \* They can encapsulate the logic of creating and configuring network objects, making the code more readable and reusable.
    - \* They can allow creating network objects at runtime, depending on the context and configuration.
  - Disadvantages:
    - \* They can introduce complexity and overhead in the code, especially if there are many types of network objects to create.
    - \* They can make the code less transparent and testable, as the client code does not know the exact type of the network object it is using.
    - \* They can violate the principle of least surprise, as the client code may not expect the behavior of the network object to vary depending on the factory method.

### Instance Methods in Networking

- Instance methods are methods that are defined on an instance of a class, such as a socket, a server, or a client.
- Instance methods can be used to perform various operations on the instance, such as sending and receiving data, closing the connection, or listening for incoming requests.
- Instance methods are usually invoked by using the dot notation, such as `socket.send(data)` or `server.accept()`.
- Some common instance methods in networking are:
  - `socket.bind(address)`: This method binds the socket to a specific address and port. The address can be a hostname, an IP address, or

a tuple of both. The port can be an integer or a string. This method is usually used by servers to specify where they listen for incoming connections.

- `socket.listen(backlog)`: This method enables the socket to accept incoming connections. The backlog argument specifies the maximum number of queued connections that can be waiting before the socket starts rejecting new ones. This method is usually used by servers after binding the socket.
- `socket.accept()`: This method waits for an incoming connection and returns a new socket object and the address of the client. This method is usually used by servers to handle each client connection in a loop or a separate thread.
- `socket.connect(address)`: This method connects the socket to a remote address and port. The address can be a hostname, an IP address, or a tuple of both. The port can be an integer or a string. This method is usually used by clients to initiate a connection with a server.
- `socket.send(data)`: This method sends data to the connected socket. The data can be a bytes object, a string, or any other object that can be converted to bytes. This method returns the number of bytes sent. This method is usually used by both clients and servers to exchange data.
- `socket.recv(bufsize)`: This method receives data from the connected socket. The bufsize argument specifies the maximum number of bytes to read. This method returns a bytes object containing the received data. This method is usually used by both clients and servers to exchange data.
- `socket.close()`: This method closes the socket and frees the resources associated with it. This method is usually used by both clients and servers to terminate the connection.

## TCP/IP Client Sockets in Networking

- TCP/IP sockets are used to implement reliable, bidirectional, persistent, point-to-point, stream-based connections between hosts on the Internet.
- TCP socket is a connection-oriented socket that uses the Transmission Control Protocol (TCP). It requires three packets to set up a connection: the SYN packet, the SYN-ACK packet, and the ACK packet.
- TCP socket is defined by the IP address of the machine and the port it uses. A socket address is the combination of an IP address and a port number.
- A socket is one endpoint of a two-way communication link between two programs running on the network. The socket mechanism provides a means of inter-process communication (IPC) by establishing named contact points between which the communication take place.
- A socket programming interface provides the routines required for inter-

process communication between applications, either on the local system or spread in a distributed, TCP/IP-based network environment.

- A TCP/IP client socket is a socket that initiates a connection to a server socket and sends or receives data over the established connection .
- A TCP/IP client socket can be created using the `Socket` class in Java or the `System.Net.Sockets.Socket` class in .NET .
- A TCP/IP client socket can use an `EndPoint` object to specify the network address of the server. It can also use a hostname and a port number to resolve the server address using the DNS service.
- A TCP/IP client socket can use the `connect` method to establish a connection to the server socket . It can also use the `beginConnect` and `endConnect` methods to perform asynchronous connection operations.
- A TCP/IP client socket can use the `getInputStream` and `getOutputStream` methods in Java or the `getStream` method in .NET to obtain the input and output streams for sending and receiving data over the connection .
- A TCP/IP client socket can use the `close` method to terminate the connection and release the resources associated with the socket . It can also use the `shutdown` method to disable sending or receiving data on the socket.

## URL in Networking

- A URL (Uniform Resource Locator) is a string of characters that identifies a resource on the Internet and provides a way to locate it.
- A URL consists of several components, such as the scheme, the authority, the path, the query, and the fragment.
- The scheme specifies the protocol used to access the resource, such as `http`, `https`, `ftp`, `mailto`, etc.
- The authority identifies the host name and the port number of the server that hosts the resource, such as `www.example.com:80`, `user@host.com`, etc.
- The path specifies the location of the resource on the server, such as `/index.html`, `/images/logo.png`, etc.
- The query is an optional part that contains additional information or parameters for the resource, such as `?q=keyword`, `?page=2`, etc. The query is separated from the path by a question mark (`?`).
- The fragment is an optional part that identifies a specific part of the resource, such as `#section1`, `#top`, etc. The fragment is separated from the rest of the URL by a hash sign (`#`).
- A URL can be absolute or relative. An absolute URL contains all the components and starts with the scheme. A relative URL omits some components and is interpreted based on the context of the base URL.
- A URL can be encoded or decoded to handle special characters or spaces. The encoding process replaces some characters with a percent sign (`%`) followed by two hexadecimal digits that represent the ASCII code of the character. The decoding process reverses the encoding process and restores the original characters.
- A URL can be normalized or canonicalized to remove any ambiguity or

redundancy. The normalization process applies some rules to simplify or standardize the URL, such as converting the scheme and the host name to lower case, removing the default port number, removing dot-segments from the path, etc.

### URL Connection in Networking

- A URL connection is an object that represents a communication link between a URL and an application.
- A URL connection can be used to read or write data from or to the resource specified by the URL, such as a web page, a file, or a database.
- A URL connection can also be used to access the metadata of the resource, such as the content type, the content length, the last modified date, and the headers.
- To create a URL connection, one needs to first create a URL object that represents the desired resource, and then call the `openConnection()` method on it. This returns a `URLConnection` object, which can be cast to a more specific subclass, such as `HttpURLConnection` or `HttpsURLConnection`, depending on the protocol of the URL.
- To read data from the resource, one can use the `getInputStream()` method of the URL connection, which returns an `InputStream` object that can be read using various methods, such as `read()`, `readLine()`, or `readAllBytes()`.
- To write data to the resource, one can use the `getOutputStream()` method of the URL connection, which returns an `OutputStream` object that can be written using various methods, such as `write()`, `writeLine()`, or `writeAllBytes()`.
- To access the metadata of the resource, one can use various methods of the URL connection, such as `getContentType()`, `getContentLength()`, `getLastModified()`, and `getHeaderField()`.
- To configure the URL connection, one can use various methods of the URL connection, such as `setConnectTimeout()`, `setReadTimeout()`, `setRequestMethod()`, `setRequestProperty()`, and `setDoOutput()`.
- To close the URL connection, one can use the `disconnect()` method of the URL connection, which releases the resources associated with the connection.

### TCP/IP Server Sockets in Networking

- A socket is an endpoint of a communication between two processes running on different machines on a network.
- A TCP/IP server socket is a socket that listens for incoming connections from TCP/IP clients on a specific port and address.
- A TCP/IP server socket can accept multiple connections from different clients, but each connection is handled by a separate socket object.



- To create a TCP/IP server socket, the following steps are required:
  1. Create a socket object using the `socket` function, specifying the address family (`AF_INET` for IPv4 or `AF_INET6` for IPv6), the socket type (`SOCK_STREAM` for TCP), and the protocol (usually 0 for default).
  2. Bind the socket object to a local address and port using the `bind` function, passing a tuple of the address and port as the argument.
  3. Make the socket object listen for incoming connections using the `listen` function, passing the maximum number of queued connections as the argument.
  4. Accept a connection from a client using the `accept` function, which returns a new socket object and the address of the client as a tuple.
  5. Communicate with the client using the new socket object, using the `send`, `sendall`, `recv`, and `close` functions to send and receive data and close the connection.
  6. Repeat steps 4 and 5 for each new connection, or use a loop or a thread to handle multiple connections concurrently.
  7. Close the server socket object using the `close` function when the server is done listening for connections.

### Datagram in Networking

- A datagram is a basic transfer unit associated with a packet-switched network.
- Datagrams are data packets which contain adequate header information so that they can be individually routed by all intermediate network switching devices to the destination.
- Datagrams provide a connectionless communication service across a packet-switched network. This means that there is no need to establish or terminate a connection before or after sending data.
- In a datagram, data is frequently divided and transmitted from source to destination without a predefined route. The order of delivery to the receiver end is not guaranteed.
- Datagrams are typically structured in header and payload sections. The header contains information such as the source and destination addresses, the length of the datagram, and the protocol type. The payload contains the actual data to be transmitted.
- Datagrams are suitable for applications that require fast and efficient data transfer, such as video streaming, voice over IP, and online gaming. However, datagrams may also suffer from drawbacks such as packet loss, duplication, delay, and reordering.

### Unit 4 - Enterprise Java Bean

- Enterprise Java Bean (EJB) is a Java API for modular construction of enterprise software.

- EJB is a server-side software component that encapsulates business logic of an application .
- EJB technology enables rapid and simplified development of distributed, transactional, secure and portable applications based on Java technology .
- There are three types of enterprise beans, entity beans, session beans, and message-driven beans.
  - Entity beans represent persistent data stored in a database.
  - Session beans represent the business logic or the actions performed by an application.
  - Message-driven beans handle asynchronous messages from a message queue or a topic.
- EJB components are hosted by an EJB container, which provides services such as security, transaction management, concurrency control, dependency injection, and lifecycle management.
- EJB components can be accessed by local or remote clients using various protocols, such as RMI, CORBA, JMS, or SOAP.
- EJB 3.0 was introduced with Java EE 5 in 2006, which simplified the development and deployment of EJB components by using annotations, dependency injection, and convention over configuration.

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write some content on the topic of Preparing a Class to be a JavaBeans. Here is what I have written:

### **Preparing a Class to be a JavaBeans**

A JavaBeans is a reusable software component that follows certain design conventions and can be manipulated visually by a tool. To prepare a class to be a JavaBeans, you need to follow these steps:

- The class must have a public, no-argument constructor. This allows the tool to instantiate the bean without any parameters.
- The class must implement the `java.io.Serializable` interface. This allows the bean to be saved and restored in a persistent state.
- The class must follow the JavaBeans naming conventions for its properties, methods, and events. A property is a public field or a pair of accessor methods (getter and setter) that expose some aspect of the bean's state. A method is a public operation that the bean can perform. An event is a notification that the bean sends to other objects when something changes in its state or behavior.
- The class can optionally implement the `java.beans.Customizer` interface. This allows the bean to provide a custom GUI for editing its properties in the tool.
- The class can optionally implement the `java.beans.PropertyChangeListener` interface. This allows the bean to listen to changes in its own or other beans' properties and react accordingly.

- The class can optionally implement the `java.beans.VetoableChangeListener` interface. This allows the bean to veto changes in its own or other beans' properties that are not acceptable.
- The class can optionally implement the `java.beans.BeanInfo` interface. This allows the bean to provide additional information about its properties, methods, and events to the tool, such as display names, descriptions, icons, etc.

Hello, I am Sydney, your AI assistant. I can help you with various tasks and topics. You have asked me to write on the topic of creating a JavaBeans. Here is the content I have generated for you:

### Creating a JavaBeans

- A JavaBeans is a reusable software component that follows certain design conventions and can be manipulated visually by a builder tool.
- To create a JavaBeans, you need to follow these steps:
  - Define a public class that implements the `java.io.Serializable` interface. This allows the bean to be saved and restored by the builder tool.
  - Provide a public no-argument constructor for the class. This allows the bean to be instantiated by the builder tool.
  - Declare the properties of the bean as private fields and provide public getter and setter methods for them. This allows the bean to expose its state to the builder tool and other components.
  - Optionally, implement the `java.beans.PropertyChangeListener` interface and provide methods to add and remove property change listeners. This allows the bean to notify other components of any changes in its properties.
  - Optionally, implement the `java.beans.VetoableChangeListener` interface and provide methods to add and remove vetoable change listeners. This allows the bean to reject any changes in its properties that are not acceptable.
  - Optionally, define one or more custom events that the bean can fire and provide methods to add and remove event listeners. This allows the bean to communicate with other components through event objects.
  - Optionally, provide a `BeanInfo` class that describes the bean's properties, events, and methods. This allows the builder tool to display more information about the bean and customize its appearance and behavior.
  - Optionally, provide a customizer class that implements the `java.beans.Customizer` interface. This allows the builder tool to provide a graphical user interface for editing the bean's properties.

## JavaBeans Properties

- JavaBeans properties are attributes of a Java class that follow a naming convention and can be accessed by methods called getters and setters.
- A property can be of any Java type, such as int, String, boolean, or an object of another class.
- A property can be read-only, write-only, or read-write, depending on the availability of the getter and setter methods.
- A property can be bound, meaning that it notifies other objects when its value changes, or constrained, meaning that it allows other objects to veto its value changes.
- A property can be indexed, meaning that it represents an array or a collection of values, and can be accessed by an index parameter in the getter and setter methods.

## Types of beans in Enterprise Java Bean

Enterprise Java Beans (EJB) are server-side components that encapsulate the business logic of an application. They are managed by an EJB container that provides services such as security, transaction management, dependency injection, concurrency control, and remote access.

There are three types of beans in EJB:

- **Session beans:** These are non-persistent beans that represent a single client-server interaction. They can be stateful, stateless, or singleton. Stateful session beans maintain the conversational state of a client across multiple requests. Stateless session beans do not maintain any state and can be pooled and reused by different clients. Singleton session beans are instantiated only once per application and provide global access to shared data or resources.
- **Message-driven beans:** These are beans that act as message consumers and process messages asynchronously from a message queue or topic. They implement the `javax.jms.MessageListener` interface and are invoked by the EJB container when a message arrives. They can be transactional and can access other EJB components or resources.
- **Entity beans:** These are persistent beans that represent the data stored in a database. They are deprecated since EJB 3.0 and replaced by Java Persistence API (JPA) entities. They can be container-managed or bean-managed. Container-managed entity beans delegate the persistence operations to the EJB container, which uses a mapping file to map the bean fields to the database columns. Bean-managed entity beans implement the persistence logic themselves, using JDBC or JPA APIs.

## Stateful Session bean in Enterprise Java Bean

- A stateful session bean is a type of enterprise bean, which preserves the conversational state with the client .

- A stateful session bean as per its name keeps associated client state in its instance variables .
- EJB Container creates a separate stateful session bean to process each client's request .
- A stateful session bean is intended for use by a single client during its lifetime and maintains a conversational state across multiple method calls .
- A stateful session bean can implement a local or remote business interface or a no-interface view .
- A stateful session bean can be annotated with `@Stateful` or declared in the deployment descriptor .
- A stateful session bean can use dependency injection to access other enterprise beans, resources, and services .
- A stateful session bean can use the `@PostConstruct` and `@PreDestroy` callbacks to perform initialization and cleanup tasks .
- A stateful session bean can use the `@PrePassivate` and `@PostActivate` callbacks to manage its state when it is passivated and activated by the container .
- A stateful session bean can use the `@Remove` annotation to mark a method that removes the bean instance from the container .
- A stateful session bean can use the `@SessionSynchronization` interface to synchronize its state with the underlying transaction context .
- A stateful session bean can be accessed concurrently by multiple threads, but the container serializes the requests to ensure thread safety .
- A stateful session bean can be clustered or replicated across multiple nodes for scalability and availability .

### Stateless Session Bean in Enterprise Java Bean

- A stateless session bean is a type of enterprise bean, which is normally used to perform independent operations .
- A stateless session bean does not have any associated client state, but it may preserve its instance state .
- A stateless session bean does not maintain any conversational state between multiple method calls by the same or different clients .
- A stateless session bean can be pooled and reused by the container to serve multiple clients, thus improving performance and scalability .
- A stateless session bean is annotated with `@Stateless` or declared in the deployment descriptor with `<session-type>Stateless</session-type>`.
- A stateless session bean can implement a business interface or a no-interface view, or both.
- A stateless session bean can access other enterprise beans, web services, and resources such as databases and JMS destinations .
- A stateless session bean can use dependency injection to obtain references to other beans or resources.
- A stateless session bean can be a web service endpoint or a message-driven

bean.

- A stateless session bean can use interceptors to add cross-cutting functionality such as logging, auditing, or security.

### **Entity bean in Enterprise Java Bean**

- An entity bean is a type of Enterprise Java Bean (EJB), which is a server-side Java EE component.
- An entity bean represents persistent data maintained in a database, such as a customer, an order, or a product.
- An entity bean can manage its own persistence (Bean managed persistence) or can delegate this function to its EJB Container (Container managed persistence).
- An entity bean is identified by a primary key, which is a unique value that distinguishes one entity bean instance from another.
- An entity bean can be accessed by multiple clients concurrently, and the EJB Container ensures the consistency and integrity of the data.
- An entity bean can implement business logic and validation rules that are related to the data it represents.
- An entity bean can be either a container-managed persistence (CMP) entity bean or a bean-managed persistence (BMP) entity bean.
  - A CMP entity bean relies on the EJB Container to handle the database access and mapping of the entity bean fields to the database columns.
  - A BMP entity bean implements the database access and mapping logic in its own code, using JDBC or other APIs.
- An entity bean can be either a session bean or a message-driven bean.
  - A session bean is a stateful or stateless component that provides a service to a client, such as processing a request or performing a calculation.
  - A message-driven bean is a stateless component that acts as a message consumer, such as receiving and processing messages from a message queue or a topic.

### **Java Database Connectivity (JDBC)**

- JDBC is the Java API that manages connecting to a database, issuing queries and commands, and handling result sets obtained from the database.
- JDBC is a Java-based data access technology used for Java database connectivity. It is part of the Java Standard Edition platform, from Oracle Corporation.
- JDBC provides universal data access from the Java programming language. Using the JDBC API, you can access virtually any data source, from relational databases to spreadsheets and flat files.
- JDBC API uses JDBC drivers to connect with the database. There are

four types of JDBC drivers: JDBC-ODBC Bridge Driver, Native Driver, Network Protocol Driver, and Thin Driver.

- A database connection URL is a string that your DBMS JDBC driver uses to connect to a database. It can contain information such as where to search for the database, the name of the database to connect to, and configuration properties. The exact syntax of a database connection URL is specified by your DBMS.

### Merging Data from Multiple Tables in JDBC

- JDBC stands for Java Database Connectivity, which is an API that allows Java programs to interact with various types of databases.
- One of the common tasks in JDBC is to merge data from multiple tables, which can be done using SQL queries that join the tables based on some common attributes or conditions.
- There are different types of joins in SQL, such as inner join, left join, right join, full join, cross join, etc. Each type of join has a different syntax and result set, depending on how the tables are related and what data is required.
- To merge data from multiple tables in JDBC, the following steps are needed:
  - Create a Connection object that represents the connection to the database.
  - Create a Statement object that can execute SQL queries on the database.
  - Write a SQL query that joins the tables using the appropriate join type and conditions.
  - Execute the query using the `executeQuery()` method of the Statement object, which returns a `ResultSet` object that contains the merged data.
  - Iterate over the `ResultSet` object using the `next()` method and the `getXXX()` methods to access the data in each row and column.
  - Close the `ResultSet`, `Statement`, and `Connection` objects to release the resources.
- Here is an example of merging data from two tables in JDBC using an inner join:

```
// Assume that the tables are as follows:
// Employee (emp_id, name, dept_id, salary)
// Department (dept_id, dept_name, location)

// Import the required packages
import java.sql.*;

public class MergeExample {
```

```

public static void main(String[] args) {
    // Declare the JDBC objects
    Connection conn = null;
    Statement stmt = null;
    ResultSet rs = null;

    try {
        // Load the JDBC driver
        Class.forName("com.mysql.cj.jdbc.Driver");

        // Establish the connection to the database
        conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/testdb", "root", "pass");

        // Create the statement object
        stmt = conn.createStatement();

        // Write the SQL query that joins the tables
        String sql = "SELECT e.name, e.salary, d.dept_name, d.location FROM Employee e INNER JOIN Department d ON e.dept_name = d.dept_name";

        // Execute the query and get the result set
        rs = stmt.executeQuery(sql);

        // Iterate over the result set and print the data
        while (rs.next()) {
            // Get the data from each column using the column name or index
            String name = rs.getString("name");
            double salary = rs.getDouble("salary");
            String dept_name = rs.getString("dept_name");
            String location = rs.getString("location");

            // Print the data
            System.out.println(name + "\t" + salary + "\t" + dept_name + "\t" + location);
        }
    } catch (Exception e) {
        // Handle any errors
        e.printStackTrace();
    } finally {
        // Close the JDBC objects
        try {
            if (rs != null) {
                rs.close();
            }
            if (stmt != null) {
                stmt.close();
            }
            if (conn != null) {
                conn.close();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```



```

        conn.close();
    }
} catch (SQLException se) {
    se.printStackTrace();
}
}
}
}
}

```

## Joining in JDBC

- Joining in JDBC is a technique to combine data from two or more tables based on a common column or condition.
- Joining in JDBC requires the use of SQL JOIN clauses, such as INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, or FULL OUTER JOIN, in the query statement.
- Joining in JDBC can be performed by creating a statement object and executing the query with the JOIN clause, and then processing the result set object that contains the joined data.
- Joining in JDBC can also be performed by using a JoinRowSet object, which is a special type of RowSet object that can hold data from multiple RowSet objects that have a match column, the column on which the JOIN is based.
- Joining in JDBC can be useful for retrieving data from related tables in a database, such as product and order tables, employee and department tables, etc.

Some examples of joining in JDBC are:

- To join the product and order tables based on the item ID column, the query statement can be:

```
SELECT * FROM Product INNER JOIN Orders ON Product.ItemID = Orders.ItemID
```

- To join the employee and department tables based on the department ID column, the query statement can be:

```
SELECT * FROM Employee LEFT OUTER JOIN Department ON Employee.DeptID = Department.DeptID
```

- To join three tables, such as product, order, and supplier tables, based on the item ID column, the query statement can be:

```
SELECT * FROM Product INNER JOIN Orders ON Product.ItemID = Orders.ItemID LEFT OUTER JOIN Supplier ON Supplier.ItemID = Product.ItemID
```

- To join two RowSet objects, such as productRowSet and orderRowSet, based on the item ID column, the JoinRowSet object can be created and populated as:

```
JoinRowSet joinRowSet = new JoinRowSetImpl();
joinRowSet.addRowSet(productRowSet, "ItemID");
```

```
joinRowSet.addRowSet(orderRowSet, "ItemID");
```

## Manipulating in JDBC

- JDBC stands for Java Database Connectivity, which is an API for connecting and executing queries on databases from Java applications.
- JDBC uses drivers to communicate with different types of databases, such as Oracle, MySQL, PostgreSQL, etc.
- JDBC allows Java applications to manipulate data in databases using SQL statements, such as SELECT, INSERT, UPDATE, DELETE, etc.
- JDBC provides classes and interfaces to perform various operations on databases, such as:
  - Establishing a connection to a database using the DriverManager or DataSource classes.
  - Creating and executing statements using the Statement, PreparedStatement, or CallableStatement classes.
  - Retrieving and processing the results of executed queries using the ResultSet class.
  - Handling exceptions and errors using the SQLException class.
  - Managing transactions and isolation levels using the Connection class.
  - Using batch updates and stored procedures for improved performance and functionality.
  - Using metadata to obtain information about the database structure and capabilities using the DatabaseMetaData and ResultSetMetaData classes.
  - Using advanced features such as scrollable and updatable result sets, rowsets, blobs, clobs, etc. using the JDBC 2.0 and JDBC 3.0 APIs.

## Databases with JDBC in JDBC

- JDBC stands for Java Database Connectivity, which is an API that allows Java programs to interact with various types of databases.
- JDBC provides a standard interface for connecting to different databases, executing SQL statements, retrieving results, and handling errors.
- JDBC consists of two main components: the JDBC driver and the JDBC API.
- The JDBC driver is a software module that implements the JDBC interface for a specific database. It acts as a bridge between the Java program and the database server. Different databases require different drivers, which can be downloaded from the database vendor's website or included in the Java Development Kit (JDK).
- The JDBC API is a set of classes and interfaces that define the methods and constants for accessing and manipulating data using JDBC. The JDBC API is part of the java.sql and javax.sql packages in the JDK.
- To use JDBC, a Java program needs to perform the following steps:

- Load the JDBC driver class using the `Class.forName()` method.
- Establish a connection to the database using the `DriverManager.getConnection()` method, which returns a `Connection` object.
- Create a `Statement` object from the `Connection` object, which can be used to execute SQL queries or updates.
- Execute the SQL statement using the `Statement.execute()`, `Statement.executeQuery()`, or `Statement.executeUpdate()` methods, which return a boolean value, a `ResultSet` object, or an `int` value, respectively.
- Process the results using the `ResultSet` object, which contains the data returned by the query. The `ResultSet` object has methods to move the cursor, get the column values, and check the metadata of the result set.
- Close the resources using the `Statement.close()`, `ResultSet.close()`, and `Connection.close()` methods, which release the memory and database resources associated with them.

### Prepared Statements in JDBC

- A prepared statement is a precompiled SQL statement that can be executed multiple times with different parameters.
- A prepared statement improves the performance and security of the database operations, as it reduces the parsing, compiling, and planning overhead, and prevents SQL injection attacks.
- A prepared statement is created by using the `prepareStatement()` method of the `Connection` interface, which takes a SQL query as a parameter and returns a `PreparedStatement` object.
- A prepared statement can have one or more parameters, which are represented by question marks (?) in the SQL query. The parameters can be set by using the `setXXX()` methods of the `PreparedStatement` interface, where XXX is the data type of the parameter.
- A prepared statement can be executed by using the `executeQuery()` or `executeUpdate()` methods of the `PreparedStatement` interface, which return a `ResultSet` or an `int` value respectively.
- A prepared statement can be reused by changing the parameters and executing it again.
- A prepared statement can be closed by using the `close()` method of the `PreparedStatement` interface, which releases the database resources associated with it.

### Transaction Processing in JDBC

- A transaction is a logical unit of work that consists of one or more SQL statements executed on a database.
- A transaction has the following properties: atomicity, consistency, isolation, and durability (ACID).

- Atomicity means that either all the statements in a transaction are executed successfully, or none of them are executed at all.
- Consistency means that a transaction preserves the integrity constraints and business rules of the database.
- Isolation means that a transaction does not interfere with other concurrent transactions, and vice versa.
- Durability means that the effects of a committed transaction are permanent and survive any system failures.
- By default, JDBC uses auto-commit mode, which means that every SQL statement is executed as a separate transaction and committed immediately.
- To disable auto-commit mode and execute multiple statements as a single transaction, use the `setAutoCommit(false)` method of the `Connection` object.
- To commit a transaction, use the `commit()` method of the `Connection` object.
- To roll back a transaction, use the `rollback()` method of the `Connection` object.
- To set the isolation level of a transaction, use the `setTransactionIsolation(int level)` method of the `Connection` object, where `level` is one of the constants defined in the `Connection` interface, such as `TRANSACTION_READ_COMMITTED` or `TRANSACTION_SERIALIZABLE`.
- To check the current isolation level of a transaction, use the `getTransactionIsolation()` method of the `Connection` object, which returns an integer value corresponding to one of the constants defined in the `Connection` interface.
- To enable or disable the savepoint feature, use the `setSavepoint()` and `releaseSavepoint()` methods of the `Connection` object, respectively.
- A savepoint is a point in a transaction that can be used to roll back a part of the transaction without affecting the rest of it.
- To create a savepoint, use the `setSavepoint()` method of the `Connection` object, which returns a `Savepoint` object.
- To roll back to a savepoint, use the `rollback(Savepoint savepoint)` method of the `Connection` object, where `savepoint` is the `Savepoint` object returned by the `setSavepoint()` method.
- To release a savepoint, use the `releaseSavepoint(Savepoint savepoint)` method of the `Connection` object, where `savepoint` is the `Savepoint` object returned by the `setSavepoint()` method.
- To handle any exceptions that may occur during transaction processing, use the `SQLException` class and its methods, such as `getErrorCode()`, `getSQLState()`, and `getMessage()`.

## Stored Procedures in JDBC

- A stored procedure is a precompiled SQL statement that can be executed by the database server.

- A stored procedure can have input parameters, output parameters, or both.
- A stored procedure can perform complex operations, such as looping, conditional logic, or error handling, that are not possible with a single SQL statement.
- A stored procedure can improve the performance and security of the database application, by reducing the network traffic and enforcing the access control.
- To execute a stored procedure from a JDBC program, the following steps are required:
  1. Create a CallableStatement object using the Connection.prepareCall() method, passing the SQL call statement as a parameter. The SQL call statement has the following syntax: `{call procedure_name [(?, ?, ...)]}`
  2. Set the values of the input parameters using the appropriate setXXX() methods of the CallableStatement object, where XXX is the data type of the parameter. The parameters are referenced by their ordinal position, starting from 1.
  3. Register the output parameters using the CallableStatement.registerOutParameter() method, specifying the parameter index, the SQL type, and optionally the scale or type name.
  4. Execute the stored procedure using the CallableStatement.execute() method.
  5. Retrieve the values of the output parameters using the appropriate getXXX() methods of the CallableStatement object, where XXX is the data type of the parameter. The parameters are referenced by their ordinal position, starting from 1.
  6. Close the CallableStatement object using the CallableStatement.close() method.

## Unit 5 - Servlets

- A servlet is a Java class that runs on a web server and handles HTTP requests and responses.
- A servlet can perform various tasks, such as generating dynamic web content, processing user input, interacting with databases, and implementing business logic.
- A servlet is managed by a servlet container, which is a component of a web server that provides the runtime environment and services for servlets.
- A servlet container is responsible for loading, initializing, executing, and destroying servlets, as well as managing their life cycle and communication.
- A servlet container also handles the mapping of URLs to servlets, the creation and management of sessions, the security and authentication of

users, and the configuration of servlets.

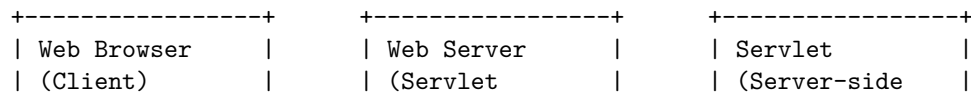
- The most widely used servlet container is Apache Tomcat, which is an open source implementation of the Java Servlet, JavaServer Pages, Java Expression Language, and Java WebSocket technologies.
- To create a servlet, one needs to extend the `javax.servlet.http.HttpServlet` class and override its methods, such as `doGet`, `doPost`, `doPut`, `doDelete`, etc., to handle different types of HTTP requests.
- A servlet can also implement the `javax.servlet.Servlet` interface directly, but this is less common and requires more coding.
- A servlet can access the request and response objects, which are instances of the `javax.servlet.http.HttpServletRequest` and `javax.servlet.http.HttpServletResponse` classes, respectively.
- The request object contains information about the client's request, such as the request method, the request URI, the request parameters, the request headers, the request body, the cookies, the session, etc.
- The response object contains information about the server's response, such as the status code, the response headers, the response body, the cookies, etc.
- A servlet can use the request and response objects to read and write data, set headers and cookies, redirect or forward requests, send errors, etc.
- A servlet can also use the `javax.servlet.ServletContext` and `javax.servlet.ServletConfig` objects, which are shared by all servlets in the same web application and by each servlet instance, respectively.
- The servlet context object contains information about the web application, such as the context path, the initialization parameters, the attributes, the resources, the log, etc.
- The servlet config object contains information about the servlet, such as the servlet name, the initialization parameters, the servlet context, etc.
- A servlet can use the servlet context and servlet config objects to access and modify the web application and servlet settings, communicate with other servlets, access web resources, etc.
- A servlet can also use the `javax.servlet.Filter` and `javax.servlet.Listener` interfaces, which are components that can intercept and modify requests and responses, and monitor and react to events in the servlet container, respectively.
- A filter is a Java class that implements the `javax.servlet.Filter` interface and overrides its `init`, `doFilter`, and `destroy` methods.
- A filter can perform various tasks, such as logging, compression, encryption, authentication, authorization, caching, etc., before or after a request is processed by a servlet or another filter.
- A filter can be mapped to a specific URL pattern, a servlet name, or a request attribute, and can be configured with initialization parameters and order of execution.
- A listener is a Java class that implements one or more of the `javax.servlet.Listener` subinterfaces, such as `ServletContextListener`, `HttpSessionListener`, `ServletRequestListener`, etc., and overrides

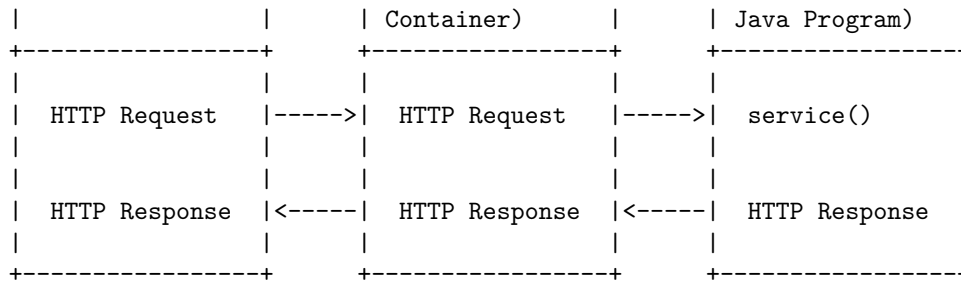
their methods.

- A listener can perform various tasks, such as initializing and destroying resources, tracking and managing sessions, logging and auditing events, etc., when a servlet context, a session, a request, or an attribute is created, modified, or destroyed.
- A listener can be registered with the servlet container using annotations or deployment descriptors, and can access the servlet context and the event objects.

## Servlet Overview and Architecture in Servlets

- Servlets are Java programs that run on a web server and handle HTTP requests and responses.
- Servlets can create dynamic web pages, process form data, interact with databases, and perform other server-side tasks.
- Servlets are portable, scalable, and efficient, as they inherit the properties of the Java language and platform.
- Servlets are managed by a servlet container, which is a component of a web server or an application server that provides the runtime environment and services for servlets.
- Servlets can be created using the `javax.servlet` and `javax.servlet.http` packages, which are part of the Java Enterprise Edition (JEE) specification.
- Servlets can implement the `Servlet` interface directly, or extend the `GenericServlet` or `HttpServlet` abstract classes, which provide convenience methods and default implementations for the servlet lifecycle methods.
- The servlet lifecycle consists of the following phases:
  - Initialization: The servlet container calls the `init()` method of the servlet once, when it is loaded into memory. The servlet can perform any one-time setup tasks in this method.
  - Request handling: The servlet container calls the `service()` method of the servlet for each HTTP request that matches the servlet's URL pattern. The servlet can read the request parameters, headers, and body, and generate the response headers, body, and status code in this method. The `service()` method can delegate the request to the `doGet()`, `doPost()`, `doPut()`, `doDelete()`, or other methods, depending on the HTTP method of the request.
  - Termination: The servlet container calls the `destroy()` method of the servlet once, when it is unloaded from memory. The servlet can perform any cleanup tasks in this method.
- The servlet architecture can be illustrated by the following diagram:





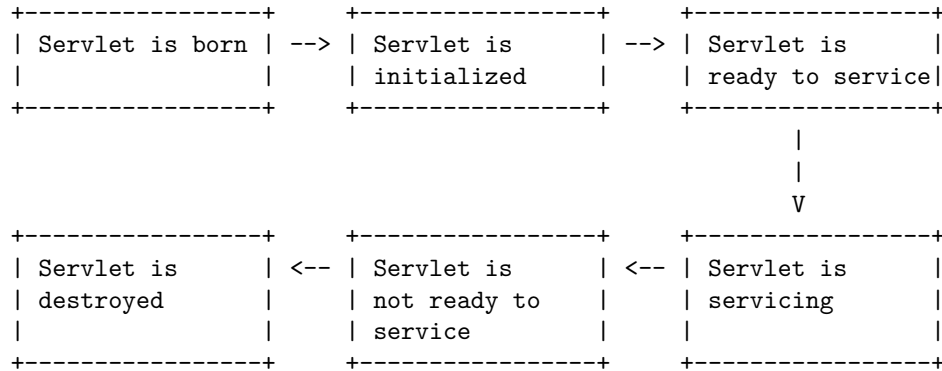
## Interface Servlet and the Servlet Life Cycle in Servlets

- The **javax.servlet.Servlet** interface is the core interface that defines a servlet. All servlets implement this interface either directly, or more commonly, by extending a class that implements the interface, such as **javax.servlet.http.HttpServlet**.
- The Servlet interface provides a **life cycle methods** to initialize a servlet, to service requests, and to remove a servlet from the server. These Servlet interface methods are central to the life cycle of a servlet .
- The life cycle of a servlet consists of the following stages:
  - **Servlet is born:** The web container loads the servlet class and creates an instance of the servlet. This happens when the servlet is requested for the first time, or when the web container is started, or when the servlet is configured with a load-on-startup element in the web.xml file.
  - **Servlet is initialized:** The web container invokes the **init()** method of the servlet. This method is used to perform any one-time initialization tasks, such as creating database connections, initializing configuration parameters, etc. The **init()** method receives a **ServletConfig** object that contains the servlet's configuration information .
  - **Servlet is ready to service:** The servlet is now ready to handle client requests. The web container creates a separate thread for each request and passes the request and response objects to the servlet .
  - **Servlet is servicing:** The web container calls the **service()** method of the servlet for each request. This method is responsible for dispatching the request to the appropriate handler method, such as **doGet()** or **doPost()** for HTTP servlets. The **service()** method also reads the request data, performs the business logic, and writes the response data .
  - **Servlet is not ready to service:** The servlet may become unavailable to service requests due to various reasons, such as configuration changes, reloading, unloading, etc. The web container will notify the servlet by calling the **isUnavailable()** method of the Servlet interface. This method returns true if the servlet is unavailable, and false otherwise.
  - **Servlet is destroyed:** The web container may decide to remove



the servlet from the server, either to free up memory resources, or to shut down the server, or to redeploy the web application. The web container will call the **destroy()** method of the servlet before removing it. This method is used to perform any finalization tasks, such as closing database connections, releasing resources, etc .

- The life cycle of a servlet is illustrated in the following diagram:



## Handling HTTP get Requests in Servlets

- A servlet is a Java class that runs on a web server and handles HTTP requests and responses.
- A servlet can handle different types of HTTP requests, such as GET, POST, PUT, DELETE, etc. by overriding the corresponding methods of the `javax.servlet.http.HttpServlet` class.
- The `doGet` method is used to handle HTTP GET requests, which are typically used to request information from the server or to send query parameters.
- The `doGet` method has two parameters: `HttpServletRequest request` and `HttpServletResponse response`. These objects represent the incoming request and the outgoing response, respectively.
- The `request` object provides methods to access the request information, such as the request URI, the query string, the headers, the cookies, the parameters, etc.
- The `response` object provides methods to set the response information, such as the status code, the headers, the cookies, the content type, the output stream, etc.
- To handle an HTTP GET request, the servlet should perform the following steps:
  - Obtain the request information from the `request` object and process it as needed.
  - Set the response information using the `response` object, such as the content type, the headers, the cookies, etc.
  - Write the response content to the output stream using the

`response.getWriter()` or `response.getOutputStream()` methods.

– Close the output stream and flush the buffer.

- The following code snippet shows an example of a servlet that handles HTTP GET requests and returns a simple HTML page:

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Get the name parameter from the request
        String name = request.getParameter("name");
        if (name == null) {
            name = "World";
        }

        // Set the content type and the character encoding of the response
        response.setContentType("text/html");
        response.setCharacterEncoding("UTF-8");

        // Get the output stream and write the response content
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head>");
        out.println("<title>Hello Servlet</title>");
        out.println("</head>");
        out.println("<body>");
        out.println("<h1>Hello, " + name + "!</h1>");
        out.println("</body>");
        out.println("</html>");

        // Close the output stream and flush the buffer
        out.close();
    }
}
```

## Handling HTTP post Requests in Servlets

- HTTP post requests are used to send data to the server, such as form inputs, file uploads, or JSON data.
- To handle HTTP post requests in servlets, you need to override the `doPost` method of the `HttpServlet` class.
- The `doPost` method takes two parameters: a `HttpServletRequest` object and a `HttpServletResponse` object.
- The `HttpServletRequest` object contains the request information, such as the request URL, headers, parameters, body, and attributes.
- The `HttpServletResponse` object contains the response information, such as the status code, headers, body, and cookies.
- To read the request parameters, you can use the `getParameter` or `getParameterValues` methods of the `HttpServletRequest` object. These methods return the values of the parameters as strings or arrays of strings, respectively.
- To read the request body, you can use the `getInputStream` or `getReader` methods of the `HttpServletRequest` object. These methods return the input stream or the reader of the request body, respectively.
- To write the response body, you can use the `getOutputStream` or `getWriter` methods of the `HttpServletResponse` object. These methods return the output stream or the writer of the response body, respectively.
- To set the response status code, you can use the `setStatus` method of the `HttpServletResponse` object. This method takes an integer value as the status code, such as 200 for OK, 404 for Not Found, or 500 for Internal Server Error.
- To set the response headers, you can use the `setHeader` or `addHeader` methods of the `HttpServletResponse` object. These methods take two strings as the header name and value, respectively. The `setHeader` method replaces any existing header with the same name, while the `addHeader` method adds a new header without replacing any existing one.
- To set the response cookies, you can use the `addCookie` method of the `HttpServletResponse` object. This method takes a `Cookie` object as the parameter. The `Cookie` object contains the cookie name, value, domain, path, expiry, and other attributes.
- To redirect the response to another URL, you can use the `sendRedirect` method of the `HttpServletResponse` object. This method takes a string as the URL to redirect to. This method sets the status code to 302 and the Location header to the URL.
- To forward the request and response to another servlet or JSP, you can use the `getRequestDispatcher` method of the `HttpServletRequest` object. This method takes a string as the path to the servlet or JSP to forward to. This method returns a `RequestDispatcher` object, which has a `forward` method that takes the request and response objects as the parameters. This method does not change the URL or the status code of the response.

Hello, I am Sydney, your AI assistant. I can help you with any topic you want to learn or discuss. You have asked me to write the content in markdown format on the topic of Redirecting Requests to Other Resources in Servlets. Here is what I have written:

## Redirecting Requests to Other Resources in Servlets

- Sometimes, a servlet may need to redirect a request to another resource, such as another servlet, a JSP page, or a static HTML file.
- There are two ways to redirect a request in servlets: using the `sendRedirect()` method of the `HttpServletResponse` interface, or using the `forward()` method of the `RequestDispatcher` interface.
- The `sendRedirect()` method instructs the client browser to send a new request to the specified URL. The original request and response objects are discarded, and the browser URL is updated with the new URL.
- The `sendRedirect()` method has the following syntax:

```
response.sendRedirect(String url);
```

- The `url` parameter can be either an absolute URL (starting with `http://` or `https://`) or a relative URL (relative to the current request URL).
- The `sendRedirect()` method is useful when the servlet wants to redirect the request to a different web server or domain, or when the servlet wants to send a response with a different status code (such as 301 Moved Permanently or 302 Found).
- The `forward()` method allows the servlet to forward the request to another resource within the same web application. The original request and response objects are preserved, and the browser URL is not changed.
- The `forward()` method has the following syntax:

```
RequestDispatcher rd = request.getRequestDispatcher(String url);  
rd.forward(request, response);
```

- The `url` parameter can be either a servlet name (starting with `/`) or a path to a JSP page or a static file (relative to the web application root).
- The `forward()` method is useful when the servlet wants to delegate the processing of the request to another resource, or when the servlet wants to include the output of another resource in its response.
- The `forward()` method can only be called before the servlet has committed the response, otherwise an `IllegalStateException` will be thrown.

## Session Tracking in Servlets

- Session tracking is a mechanism that allows a web server to maintain the state of a user across multiple HTTP requests.
- A session is a collection of data associated with a specific user, such as username, preferences, shopping cart items, etc.

- A session is created when a user first visits a web site or application, and is destroyed when the user leaves or logs out.
- Session tracking is useful for implementing features such as authentication, personalization, and e-commerce.
- There are four main techniques for session tracking in servlets: cookies, URL rewriting, hidden form fields, and HttpSession objects.
- Cookies are small pieces of data that are stored by the browser and sent to the server with every request. Cookies can store session identifiers, user preferences, or other information. Cookies are easy to use, but have some limitations, such as size, security, and browser compatibility.
- URL rewriting is a technique that appends the session identifier to every URL in the web page. For example, `http://example.com/servlet?name=John&sessionID=123456`. URL rewriting does not rely on the browser, but it can be cumbersome, insecure, and affect the usability of the web site.
- Hidden form fields are input elements in HTML forms that are not visible to the user, but can store session data. For example, `<input type="hidden" name="sessionID" value="123456">`. Hidden form fields can only be used with forms that use the POST method, and they can be tampered with by the user.
- HttpSession objects are Java objects that are created and managed by the servlet container. HttpSession objects can store any type of data as attributes, and are accessible by any servlet in the same web application. HttpSession objects are the most convenient and secure way of session tracking in servlets, but they require more memory and processing power on the server.

## Cookies in Servlets

- Cookies are small pieces of data that are stored by a web browser on the client side.
- Cookies are used to store information about the user's preferences, session state, authentication, etc.
- Cookies are sent by the server to the browser using the **Set-Cookie** header in the HTTP response.
- Cookies are sent back by the browser to the server using the **Cookie** header in the HTTP request.
- Cookies have a name, a value, and some optional attributes, such as **Domain**, **Path**, **Expires**, **Secure**, and **HttpOnly**.
- Cookies can be created, read, updated, and deleted by servlets using the `javax.servlet.http.Cookie` class and the `javax.servlet.http.HttpServletRequest` and `javax.servlet.http.HttpServletResponse` interfaces.
- To create a cookie, a servlet can use the **Cookie** constructor with the name and value of the cookie as parameters, and then call the `addCookie`

method of the `HttpServletResponse` object to send the cookie to the browser.

- To read a cookie, a servlet can use the `getCookies` method of the `HttpServletRequest` object to get an array of `Cookie` objects, and then loop through the array to find the cookie with the desired name.
- To update a cookie, a servlet can use the `setValue` method of the `Cookie` object to change the value of the cookie, and then call the `addCookie` method of the `HttpServletResponse` object to send the updated cookie to the browser.
- To delete a cookie, a servlet can use the `setMaxAge` method of the `Cookie` object to set the expiration time of the cookie to zero, and then call the `addCookie` method of the `HttpServletResponse` object to send the expired cookie to the browser.

### Session Tracking with Http Session in Servlets

- Session tracking is a mechanism to maintain the state of a user across multiple requests.
- Http session is one of the techniques to implement session tracking in servlets.
- Http session is an object that is created by the servlet container and associated with a unique session ID.
- Http session can store any type of data as attributes, which can be accessed by any servlet that belongs to the same session.
- Http session is maintained by using cookies, URL rewriting, or hidden form fields.
- Http session has a predefined timeout period, after which it is invalidated by the servlet container.
- Http session provides methods to create, access, modify, and invalidate the session and its attributes. Some of the methods are:
  - `HttpSession getSession(boolean create)`: Returns the current session associated with the request, or creates a new session if none exists and the create parameter is true.
  - `String getId()`: Returns the unique session ID of the session.
  - `Object getAttribute(String name)`: Returns the value of the specified attribute from the session, or null if no such attribute exists.
  - `void setAttribute(String name, Object value)`: Sets the value of the specified attribute in the session, or removes the attribute if the value is null.
  - `void removeAttribute(String name)`: Removes the specified attribute from the session.

- `void invalidate()`: Invalidates the session and unbinds any objects bound to it.

### Java Server Pages (JSP) in Servlets

- Java Server Pages (JSP) are a technology that allows web developers to create dynamic web pages using HTML, XML, or other document types.
- JSP are similar to servlets, but they have a simpler syntax and can embed Java code directly into the web page.
- JSP are compiled into servlets by a JSP engine, which is part of a web server or a web container.
- JSP have access to the same objects and methods as servlets, such as request, response, session, application, out, etc.
- JSP can also use custom tags, which are reusable components that encapsulate Java code or other JSP elements.
- JSP can be used for various purposes, such as generating HTML, XML, JSON, or other formats, processing user input, connecting to databases, implementing business logic, etc.
- JSP have some advantages over servlets, such as:
  - They separate presentation from logic, making the code more readable and maintainable.
  - They allow web designers and developers to work together more easily, as they can use different tools and languages.
  - They support rapid development and testing, as they do not require recompilation or redeployment for every change.
  - They can use various frameworks and libraries, such as JavaServer Faces (JSF), Spring MVC, Struts, etc.

### Introduction to JSP in Servlets

- JSP stands for JavaServer Pages, which is a technology that allows web developers to create dynamic web pages using Java code.
- JSP is based on the servlet technology, which is a Java class that runs on a web server and handles HTTP requests and responses.
- JSP files are similar to HTML files, but they can also contain Java code snippets, called scriptlets, that are executed on the server side.
- JSP files have the extension `.jsp` and are compiled into servlets by the web server when they are requested by a client.
- JSP files can also use special tags, called JSP tags, that provide additional functionality, such as accessing JavaBeans, connecting to databases, or including other JSP files.
- JSP files can also use custom tags, which are user-defined tags that encapsulate reusable logic or content.
- JSP files can also use expression language (EL), which is a simple syntax that allows accessing and manipulating data in Java objects or web components.

- JSP files can also use JSP standard tag library (JSTL), which is a collection of predefined tags that provide common functionality, such as iteration, conditional, formatting, or internationalization.

## Java Server Pages Overview in Servlets

- Java Server Pages (JSP) are a technology that allows web developers to create dynamic web pages using HTML, XML, or other document types.
- JSP are similar to servlets, but they have a simpler syntax and can embed Java code directly into the web page.
- JSP are compiled into servlets by a JSP engine, which is part of a web server or a web container.
- JSP have access to the same objects and methods as servlets, such as request, response, session, application, out, etc.
- JSP can also use custom tags, which are reusable components that encapsulate Java code or other JSP elements.
- JSP have a life cycle that consists of the following phases:
  - Translation: The JSP engine translates the JSP page into a servlet class.
  - Compilation: The JSP engine compiles the servlet class into a byte-code file.
  - Loading: The web container loads the servlet class into memory.
  - Initialization: The web container invokes the `init()` method of the servlet class.
  - Execution: The web container invokes the `service()` method of the servlet class for each request.
  - Destruction: The web container invokes the `destroy()` method of the servlet class when it is no longer needed.
- JSP can be divided into two types: scriptlet and expression.
  - Scriptlet: A scriptlet is a block of Java code that is executed by the JSP engine. It can contain any valid Java statements, such as variable declarations, assignments, loops, conditionals, etc. A scriptlet is enclosed by `<%` and `%>` tags.
  - Expression: An expression is a Java expression that is evaluated by the JSP engine and the result is inserted into the output stream. It can contain any valid Java expression, such as literals, operators, method calls, etc. An expression is enclosed by `<%=` and `%>` tags.
- JSP can also include directives, comments, and declarations.
  - Directive: A directive is an instruction to the JSP engine that affects the behavior of the JSP page. It can specify the page attributes, the import statements, the tag libraries, etc. A directive is enclosed by `<%@` and `%>` tags.
  - Comment: A comment is a remark that is ignored by the JSP engine. It can be used to document the JSP page or to hide some code from the JSP engine. A comment is enclosed by `<%-` and `-%>` tags.
  - Declaration: A declaration is a block of Java code that is executed



only once when the JSP page is translated into a servlet. It can contain variable or method declarations that are accessible throughout the JSP page. A declaration is enclosed by `<%!` and `%>` tags.

## **A First Java Server Page Example in Servlets**

- A Java Server Page (JSP) is a web page that contains Java code embedded in HTML or XML.
- A JSP is compiled into a servlet by a JSP compiler, which is a special type of servlet container.
- A servlet is a Java class that handles HTTP requests and generates HTTP responses.
- A servlet can access the request and response objects, as well as other objects that provide information about the web application and the server environment.
- A servlet can also use JavaBeans, custom tags, expression language, and other features to simplify the web development process.
- A JSP can be seen as a convenient way of writing servlets, as it allows the separation of presentation logic (HTML or XML) and business logic (Java code).
- A JSP can also be seen as a template engine, as it can dynamically generate web pages based on the data provided by the servlet or other sources.
- A JSP follows a specific syntax and structure, which includes directives, declarations, scriptlets, expressions, comments, and actions.
- A JSP directive is a statement that provides instructions to the JSP compiler or the servlet container, such as the page language, the content type, the import statements, etc.
- A JSP declaration is a statement that declares variables or methods that can be used in the JSP.
- A JSP scriptlet is a block of Java code that is executed when the JSP is processed by the servlet container.
- A JSP expression is a Java expression that is evaluated and inserted into the output stream of the JSP.
- A JSP comment is a comment that is ignored by the JSP compiler and the servlet container, but can be seen in the source code of the JSP.
- A JSP action is a tag that invokes a built-in or custom functionality, such as including another JSP, forwarding the request to another resource, setting or getting a JavaBean property, etc.

- A simple JSP example that displays the current date and time is shown below:

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head>
<title>A First JSP Example</title>
</head>
<body>
<h1>A First JSP Example</h1>
<p>The current date and time is: <%= new java.util.Date() %></p>
</body>
</html>
```

- The first line of the JSP is a directive that specifies the page language, the content type, and the character encoding of the JSP.
- The HTML tags define the structure and appearance of the web page.
- The JSP expression `<%= new java.util.Date() %>` is evaluated and inserted into the output stream of the JSP, displaying the current date and time.

## Implicit Objects in Servlets

- Implicit objects are Java objects that are created by the servlet container and are available to all the servlets within a web application.
- Implicit objects are also known as pre-defined variables or pre-defined objects.
- Implicit objects are used to access the information related to a particular request, response, application, session, page, or exception.
- Implicit objects are part of the Java Servlet API and are defined in the `javax.servlet` and `javax.servlet.jsp` packages.
- There are nine implicit objects in servlets: `request`, `response`, `out`, `session`, `application`, `config`, `page`, `pageContext`, and `exception`.
- The `request` object represents the HTTP request from the client. It contains the request parameters, headers, cookies, attributes, and other information.
- The `response` object represents the HTTP response to the client. It allows the servlet to set the response status, headers, cookies, and content.
- The `out` object is a `PrintWriter` object that can be used to write the response content to the client. It has methods to print text, HTML, or XML.
- The `session` object represents the HTTP session associated with the request. It allows the servlet to store and retrieve session attributes across multiple requests from the same client.
- The `application` object represents the servlet context of the web application. It allows the servlet to access the initialization parameters, attributes, and resources of the application.

- The **config** object represents the servlet configuration object. It allows the servlet to access the initialization parameters and other information specific to the servlet.
- The **page** object is a reference to the current servlet instance. It can be used to invoke the methods of the servlet.
- The **pageContext** object encapsulates the information related to the current page. It provides access to the other implicit objects, as well as the request and response objects.
- The **exception** object represents the exception thrown by the servlet or the JSP page. It contains the exception message, type, stack trace, and other information. It is only available in the error pages.

### Scripting in Servlets

- Scripting in servlets refers to the use of scripting elements such as expressions, scriptlets, and declarations to embed Java code within a JSP page.
- Scripting elements are delimited by special tags that start with `<%` and end with `%>`.
- Expressions are used to evaluate a Java expression and insert its value into the output stream. They have the syntax `<%= expression %>`.
- Scriptlets are used to execute a block of Java code that does not produce any output. They have the syntax `<% code %>`.
- Declarations are used to declare variables and methods that can be used by other scripting elements. They have the syntax `<%! declaration %>`.
- Scripting elements can access implicit objects that are predefined by the JSP container, such as **request**, **response**, **session**, **out**, etc.
- Scripting elements can also use JSP directives and actions to control the behavior and structure of the JSP page.
- Scripting elements are processed by the JSP container at runtime and translated into Java code that is part of the servlet class generated from the JSP page.
- Scripting elements should be used sparingly and with caution, as they can reduce the readability, maintainability, and reusability of the JSP page. They can also introduce security risks if they are not properly validated and sanitized.

### Standard Actions in Servlets

- Standard actions are predefined tags that perform some common tasks in JSP pages.
- They have the syntax `<jsp:action_name attribute="value" />` where **action\_name** is the name of the standard action and **attribute** is a parameter that modifies the behavior of the action.
- Some of the standard actions are:

- **<jsp:include>**: This action includes the content of another resource (such as a JSP page, an HTML file, or a servlet) at the time of request processing. It has two attributes: **page** and **flush**. The **page** attribute specifies the relative URL of the resource to be included, and the **flush** attribute indicates whether the output buffer should be flushed before including the resource. The default value of **flush** is **false**.
- **<jsp:forward>**: This action forwards the request to another resource (such as a JSP page, an HTML file, or a servlet) for further processing. It has one attribute: **page**. The **page** attribute specifies the relative URL of the resource to be forwarded to. The **<jsp:forward>** action terminates the current JSP page and discards any output generated so far.
- **<jsp:param>**: This action adds a parameter to the query string of the request. It is used inside the **<jsp:include>** or **<jsp:forward>** actions to pass additional information to the included or forwarded resource. It has two attributes: **name** and **value**. The **name** attribute specifies the name of the parameter, and the **value** attribute specifies the value of the parameter.
- **<jsp:plugin>**: This action generates the necessary HTML code to invoke a Java applet or a JavaBean component in the browser. It has two attributes: **type** and **code**. The **type** attribute specifies whether the plugin is an applet or a bean, and the **code** attribute specifies the name of the class file or the JAR file that contains the plugin. The **<jsp:plugin>** action also supports nested **<jsp:params>** actions to pass parameters to the plugin.
- **<jsp:useBean>**: This action creates or locates a JavaBean component and associates it with a scripting variable. It has three attributes: **id**, **class**, and **scope**. The **id** attribute specifies the name of the scripting variable, the **class** attribute specifies the fully qualified name of the JavaBean class, and the **scope** attribute specifies the visibility of the bean. The possible values of **scope** are **page**, **request**, **session**, or **application**. The **<jsp:useBean>** action also supports nested **<jsp:setProperty>** and **<jsp:getProperty>** actions to set and get the properties of the bean.
- **<jsp:setProperty>**: This action sets the value of a property of a JavaBean component. It has three attributes: **name**, **property**, and **value**. The **name** attribute specifies the name of the scripting variable that refers to the bean, the **property** attribute specifies the name of the property to be set, and the **value** attribute specifies the value of the property. Alternatively, the **value** attribute can be replaced by a nested **<jsp:param>** action to set the property value from a request parameter.
- **<jsp:getProperty>**: This action gets the value of a property of a JavaBean component and writes it to the output stream. It has two attributes: **name** and **property**. The **name** attribute specifies

the name of the scripting variable that refers to the bean, and the **property** attribute specifies the name of the property to be retrieved.

## Directives in Servlets

- Directives are instructions that control the structure and properties of a servlet.
- Directives are not part of the servlet code, but are placed at the top of the source file, before the package declaration.
- Directives are enclosed in `<%@` and `%>` tags.
- There are three types of directives in servlets: page, include, and taglib.
- The page directive defines the attributes of the servlet, such as the content type, the language, the error page, the session management, etc.
- The syntax of the page directive is:

`<%@ page attribute="value" %>`

- Multiple attributes can be specified in a single page directive, separated by commas.
- Some of the common attributes of the page directive are:
  - `contentType`: specifies the MIME type and the character encoding of the servlet output. The default value is `"text/html; charset=ISO-8859-1"`.
  - `import`: specifies the packages or classes that need to be imported in the servlet. Multiple packages or classes can be specified, separated by commas.
  - `errorPage`: specifies the URL of another servlet or JSP page that handles the exceptions thrown by the current servlet. The error page must have the `isErrorPage` attribute set to true.
  - `isErrorPage`: specifies whether the current servlet is an error page or not. The default value is false. If true, the servlet can access the exception object using the implicit variable `exception`.
  - `session`: specifies whether the servlet participates in session tracking or not. The default value is true. If false, the servlet cannot access the implicit object `session`.
  - `isThreadSafe`: specifies whether the servlet can handle multiple requests concurrently or not. The default value is true. If false, the servlet is synchronized, which means only one thread can execute it at a time.
  - `buffer`: specifies the size of the output buffer used by the servlet. The default value is 8kb. If none, no buffering is used.
  - `autoFlush`: specifies whether the output buffer is automatically flushed when full or not. The default value is true. If false, an

exception is thrown when the buffer is full.

- language: specifies the scripting language used in the servlet. The default value is “java”.

- The include directive includes the content of another file, such as HTML, JSP, or text, in the current servlet.
- The syntax of the include directive is:

```
<%@ include file="filename" %>
```

- The file attribute specifies the relative path of the file to be included.
- The include directive is processed at translation time, which means the content of the included file is copied into the servlet source code before compilation.
- The include directive is different from the include action (`<include>`), which is processed at request time, and can include dynamic content generated by another servlet or JSP page.
- The taglib directive declares the use of a custom tag library in the servlet.
- The syntax of the taglib directive is:

```
<%@ taglib uri="uri" prefix="prefix" %>
```

- The uri attribute specifies the location of the tag library descriptor (TLD) file, which defines the custom tags and their attributes.
- The prefix attribute specifies the prefix that is used to invoke the custom tags in the servlet.
- The taglib directive enables the use of custom tags that can simplify the servlet code and provide reusable functionality.

### Custom Tag Libraries in Servlets

- Custom tag libraries are a way of creating reusable components in JavaServer Pages (JSP) that can be invoked by using a special syntax in the JSP page.
- Custom tag libraries are defined by two components: tag handlers and tag library descriptors.
- Tag handlers are Java classes that implement the `javax.servlet.jsp.tagext.Tag` interface or one of its subclasses. They contain the logic and behavior of the custom tag.
- Tag library descriptors are XML files that describe the attributes, body content, and scripting variables of the custom tags. They also map the tag names to the corresponding tag handler classes.
- Custom tag libraries can be packaged as JAR files and deployed in the `WEB-INF/lib` directory of the web application. They can also be placed in the `/WEB-INF/tags` directory of the web application or in a subdirectory of it.

- Custom tag libraries can be used in a JSP page by declaring a taglib directive that specifies the URI of the tag library descriptor. For example:

```
<%@ taglib uri="http://example.com/mytags" prefix="my" %>
```

- The prefix attribute defines a namespace for the custom tags in the JSP page. The custom tags can then be invoked by using the prefix and the tag name. For example:

```
<my:hello name="John" />
```

- Custom tag libraries can provide various benefits, such as:
  - Encapsulating complex or repetitive logic in a reusable component.
  - Separating presentation from business logic.
  - Enhancing the readability and maintainability of the JSP page.
  - Extending the functionality and features of the JSP technology.